

Лекция 9. Бинарные кучи

E-mail: lazycoyote11@gmail.com

*Курс «Структуры и алгоритмы обработки данных»
Весенний семестр, 2026 г.*

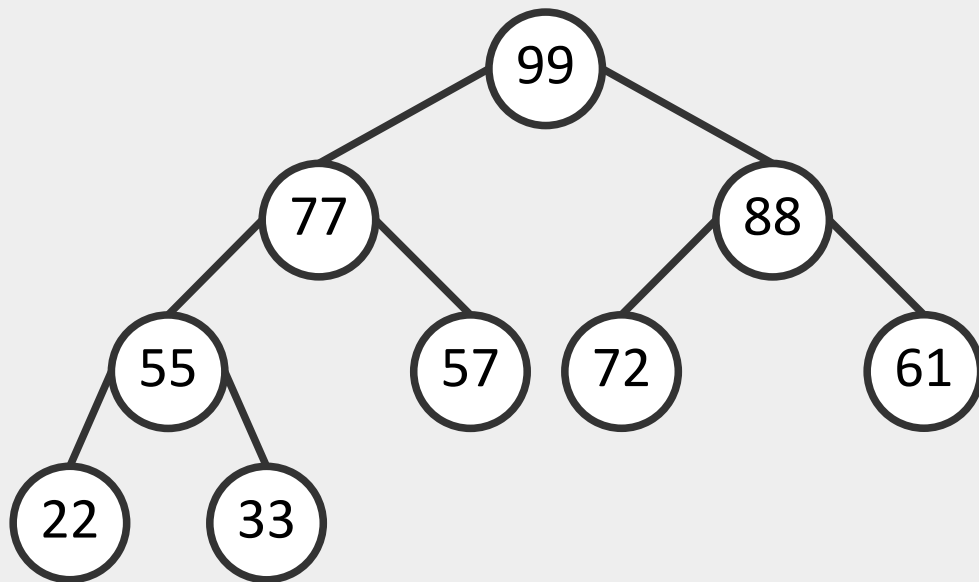
АТД «Очередь с приоритетом» (priority queue)

- **Очередь с приоритетом** (*priority queue*) – очередь, в которой элементы имеют приоритет (вес)
- Первым извлекается элемент с наибольшим приоритетом (ключом)
- **Поддерживаемые операции:**
 - **Insert** – добавление элемента в очередь
 - **Max** – возвращает элемент с максимальным приоритетом
 - **ExtractMax** – удаляет из очереди элемент с максимальным приоритетом
 - **IncreaseKey** – изменяет значение приоритета заданного элемента
 - **Merge** – сливает две очереди в одну

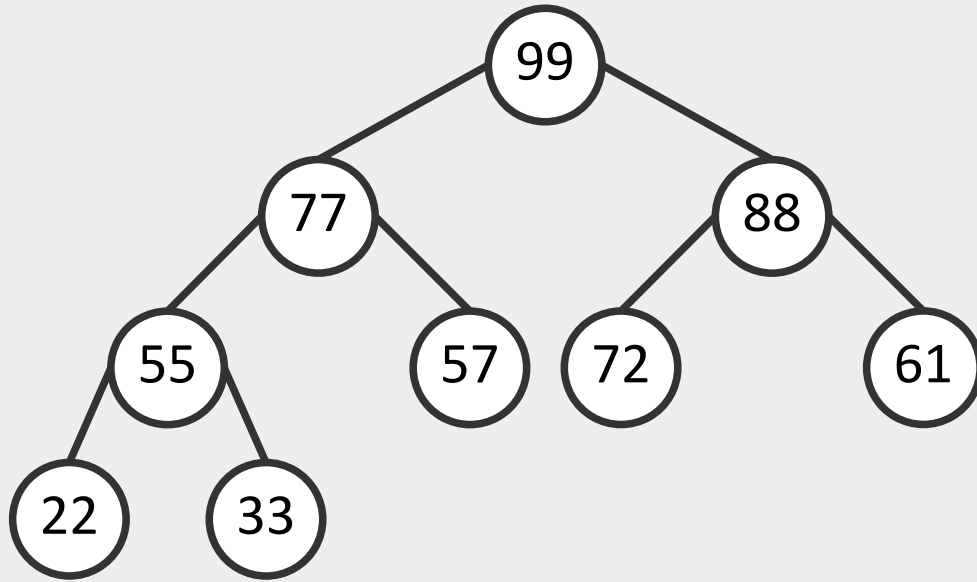
Значение (value)	Приоритет (priority)
Кот	34
Волк	45
Рысь	21

Бинарная куча (binary heap)

- **Бинарная куча** (пирамида, сортирующее дерево, binary heap) – это двоичное дерево, удовлетворяющее следующим условиям:
- Приоритет любой вершины не меньше ($\geq$) приоритета ее потомков
- Дерево является **полным бинарным деревом** (*complete binary tree*) – все уровни заполнены слева направо (возможно, за исключением последнего)

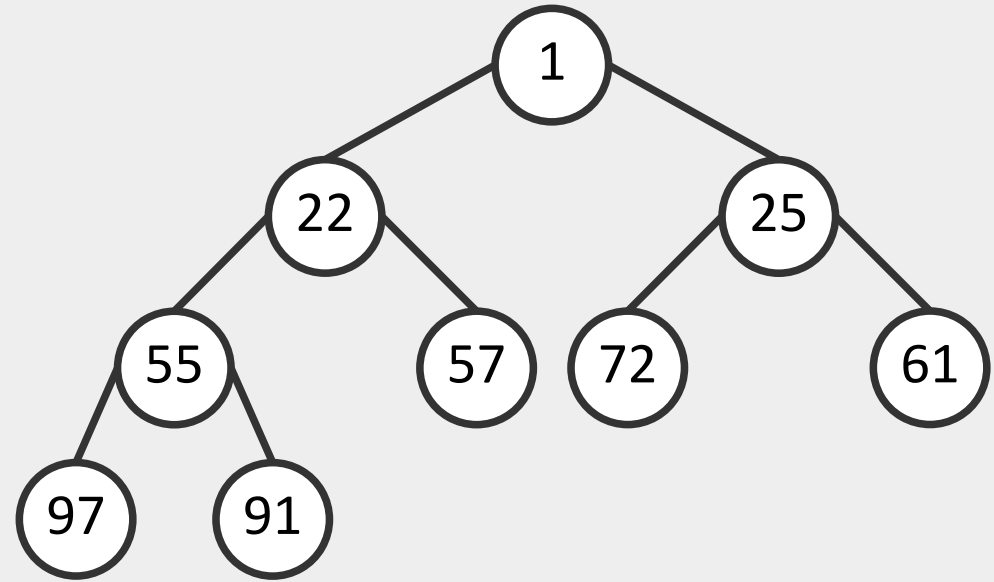


Бинарная куча (binary heap)



Невозрастающая куча
max-heap

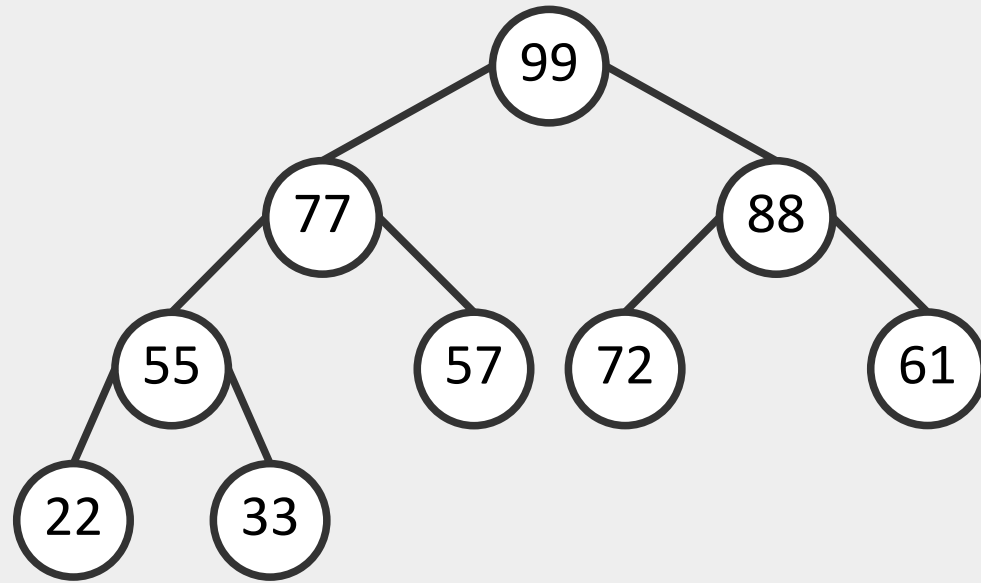
Приоритет любой вершины
не меньше ($\geq$) приоритета потомков



Неубывающая куча
min-heap

Приоритет любой вершины
не больше ($\leq$) приоритета потомков

Реализация бинарной кучи на основе массива



max-heap, 9 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	77	88	55	57	72	61	22	33	...

- Корень дерева хранится в ячейке $H[0]$ – максимальный элемент
- Индекс родителя узла i : **$Parent(i) = \lfloor (i - 1) / 2 \rfloor$**
- Индекс левого дочернего узла: **$Left(i) = 2i + 1$**
- Индекс правого дочернего узла: **$Right(i) = 2i + 2$**
- $H[Parent(i)] \geq H[i]$

Реализация бинарной кучи на основе массива

```
struct heapnode {  
    int key;      /* Приоритет (ключ) */  
    char *value; /* Значение */  
};  
  
struct heap {  
    int maxsize; /* Максимальный размер кучи */  
    int nnodes;  /* Число элементов */  
    struct heapnode *nodes; /* Массив узлов [1...maxsize] */  
};
```

Создание пустой кучи

```
struct heap *heap_create(int maxsize)
{
    struct heap *h;
    h = malloc(sizeof(*h));
    if (h != NULL) {
        h->maxsize = maxsize;
        h->nnodes = 0;
        /* Последний индекс - maxsize */
        h->nodes = malloc(sizeof(*h->nodes) * (maxsize + 1));
        if (h->nodes == NULL) {
            free(h);
            return NULL;
        }
    }
    return h;
}
```

$$T_{\text{Create}} = O(1)$$

Удаление кучи

```
void heap_free(struct heap *h)
{
    free(h->nodes);
    free(h);
}
```

$$T_{\text{Free}} = O(1)$$

```
void heap_swap(struct heapnode *a, struct heapnode *b)
{
    struct heapnode temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

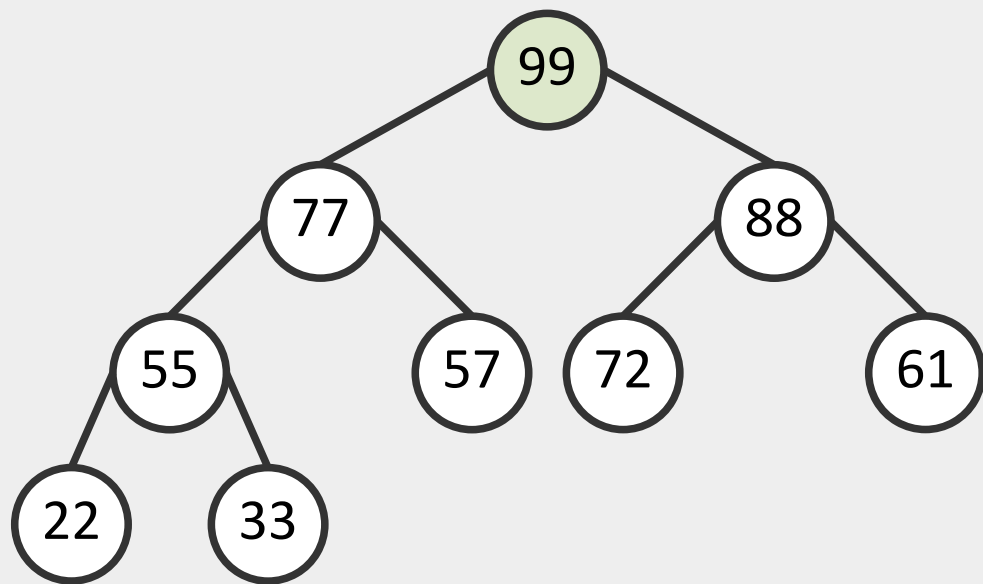
$$T_{\text{Swap}} = O(1)$$

Поиск максимального элемента

```
struct heapnode *heap_max(struct heap *h)
{
    if (h->nnodes == 0)
        return NULL;
    return &h->nodes[1];
}
```

$$T_{\text{Max}} = O(1)$$

Поиск максимального элемента

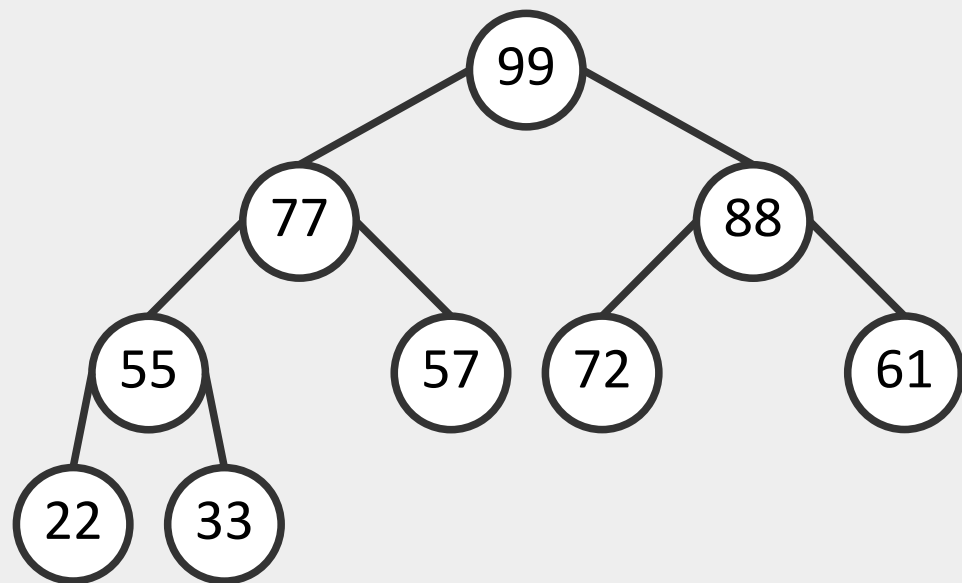


max-heap, 9 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	77	88	55	57	72	61	22	33	...

- Корень дерева хранится в ячейке $H[0]$ – максимальный элемент
- Индекс родителя узла i : $\text{Parent}(i) = \lfloor (i - 1) / 2 \rfloor$
- Индекс левого дочернего узла: $\text{Left}(i) = 2i + 1$
- Индекс правого дочернего узла: $\text{Right}(i) = 2i + 2$
- $H[\text{Parent}(i)] \geq H[i]$

Вставка элемента в бинарную кучу

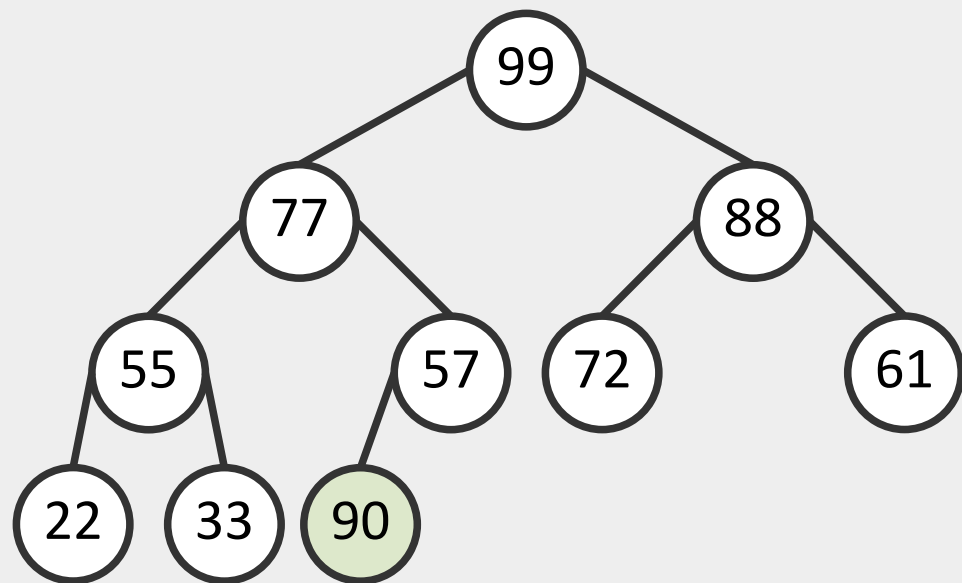


- Вставка элемента с приоритетом 90

max-heap, 9 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	77	88	55	57	72	61	22	33	...

Вставка элемента в бинарную кучу

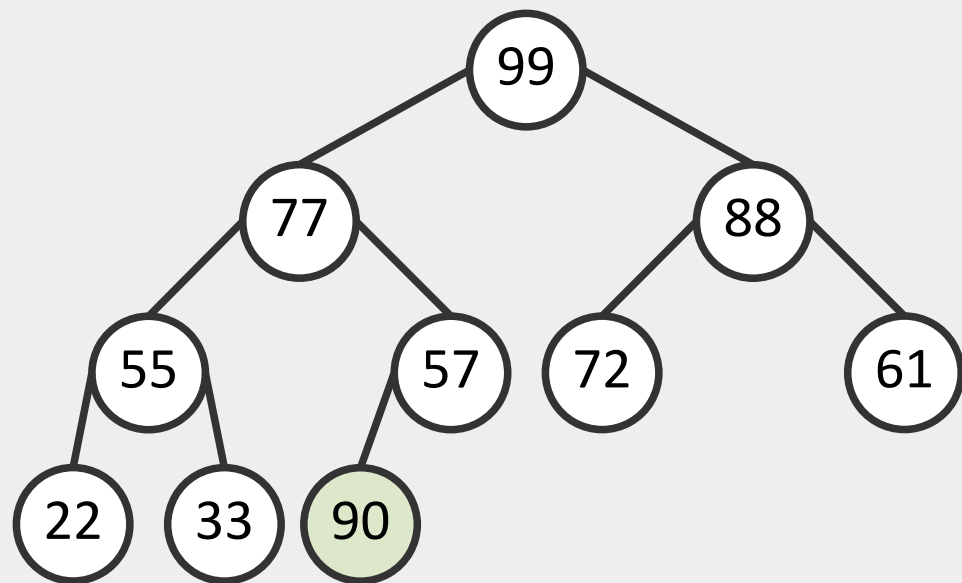


max-heap, 9 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	77	88	55	57	72	61	22	33	90

- Вставка элемента с приоритетом 90
- Добавляется на **текущий уровень** или на новый, если текущий заполнен

Вставка элемента в бинарную кучу

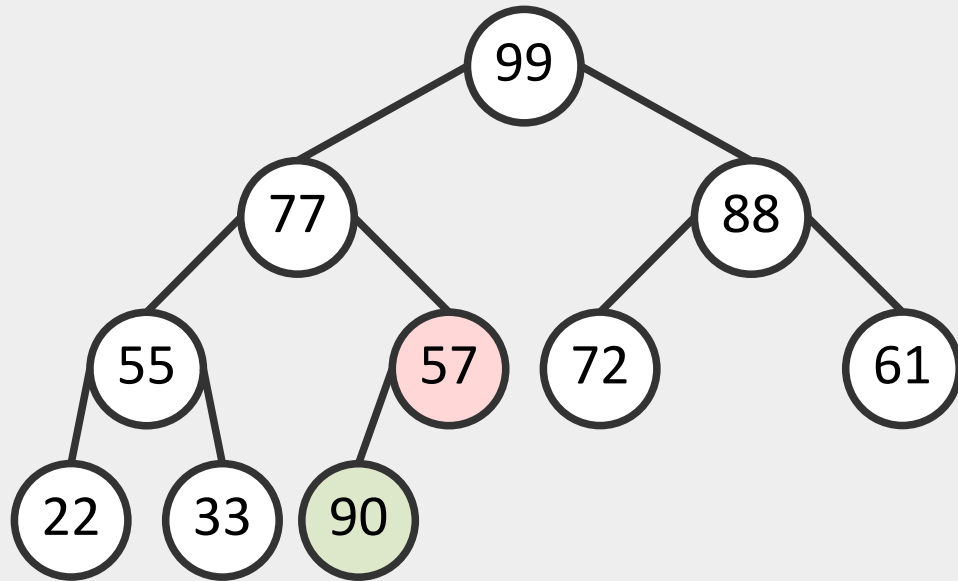


max-heap, 9 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	77	88	55	57	72	61	22	33	90

- Вставка элемента с приоритетом 90
- Добавляется на **текущий уровень** или на новый, если текущий заполнен
- **HeapifyUp** (двигаем элемент вверх по куче, если свойства нарушены)

Вставка элемента в бинарную кучу

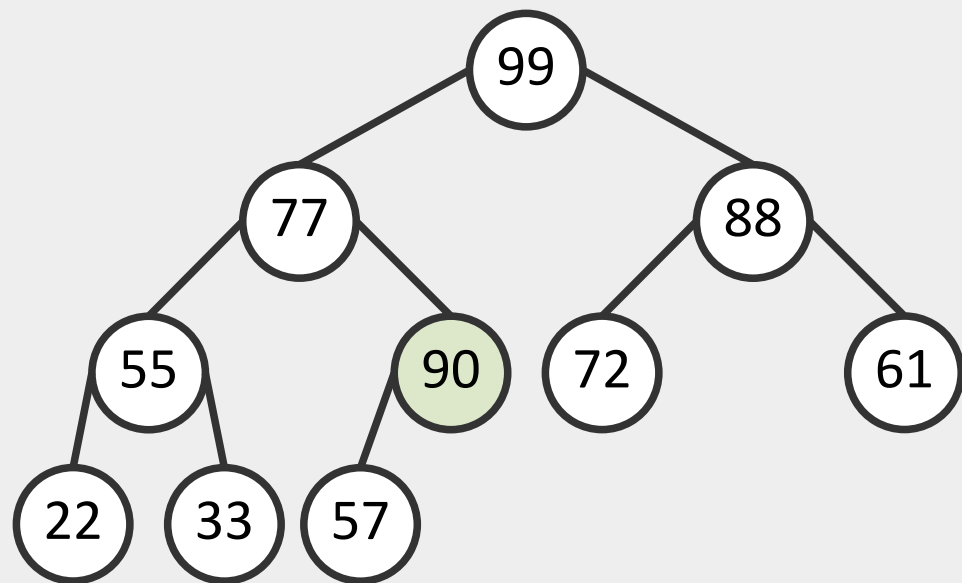


max-heap, 9 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	77	88	55	57	72	61	22	33	90

- Вставка элемента с приоритетом 90
- Добавляется на **текущий уровень** или на новый, если текущий заполнен
- **HeapifyUp** (двигаем элемент вверх по куче, если свойства нарушены)

Вставка элемента в бинарную кучу

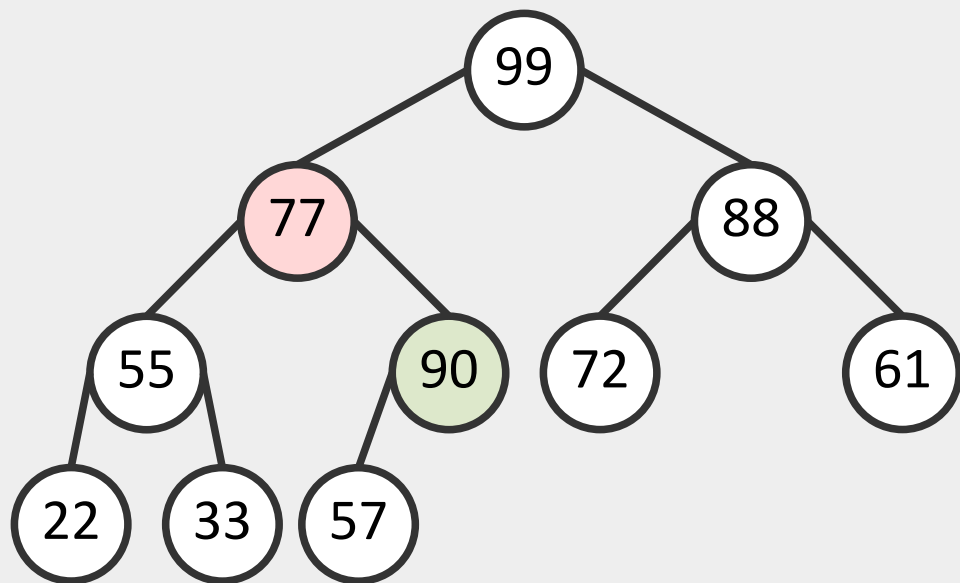


max-heap, 9 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	77	88	55	90	72	61	22	33	57

- Вставка элемента с приоритетом 90
- Добавляется на **текущий уровень** или на новый, если текущий заполнен
- **HeapifyUp** (двигаем элемент вверх по куче, если свойства нарушены)

Вставка элемента в бинарную кучу

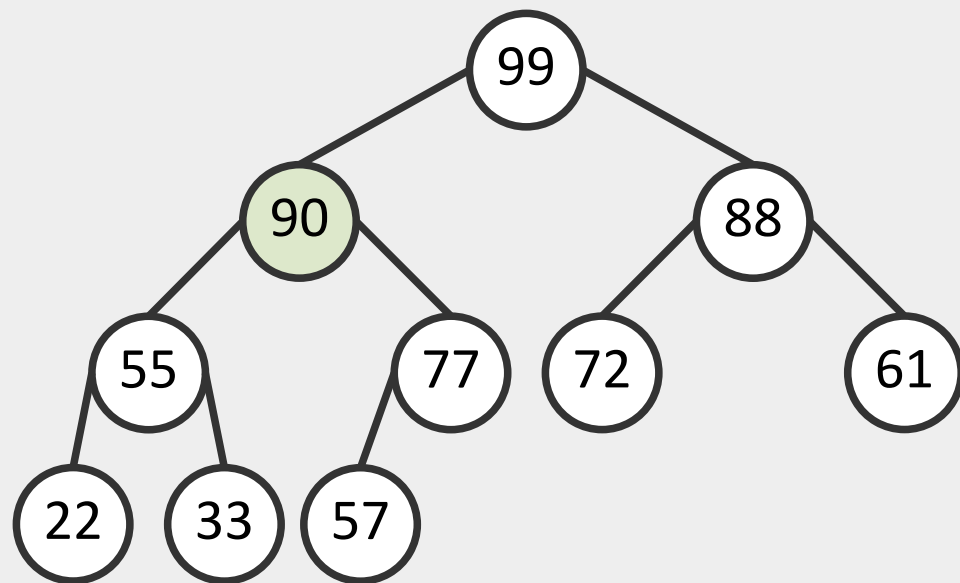


max-heap, 9 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	77	88	55	90	72	61	22	33	57

- Вставка элемента с приоритетом 90
- Добавляется на **текущий уровень** или на новый, если текущий заполнен
- **HeapifyUp** (двигаем элемент вверх по куче, если свойства нарушены)

Вставка элемента в бинарную кучу

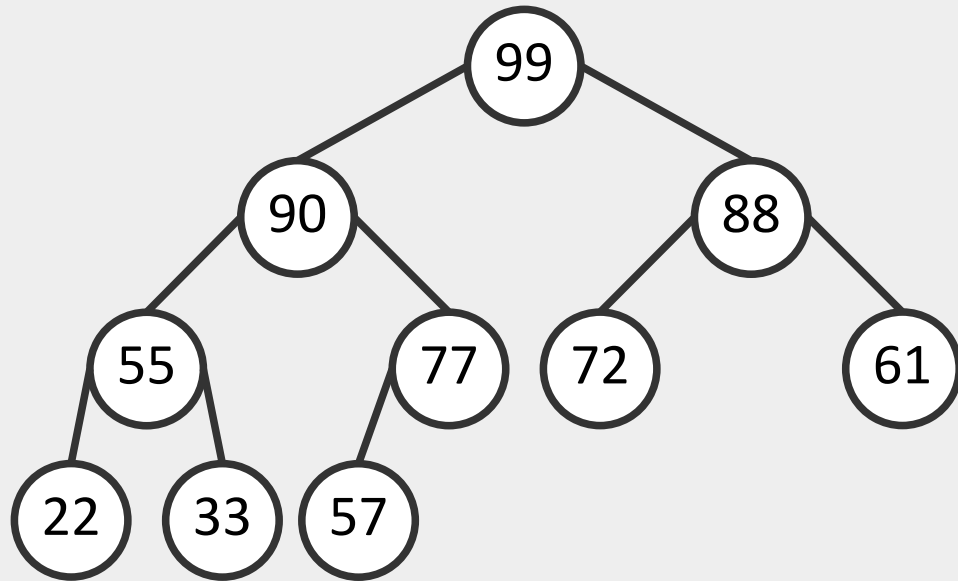


max-heap, 9 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	90	88	55	77	72	61	22	33	57

- Вставка элемента с приоритетом 90
- Добавляется на **текущий уровень** или на новый, если текущий заполнен
- **HeapifyUp** (двигаем элемент вверх по куче, если свойства нарушены)

Вставка элемента в бинарную кучу



max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	90	88	55	77	72	61	22	33	57

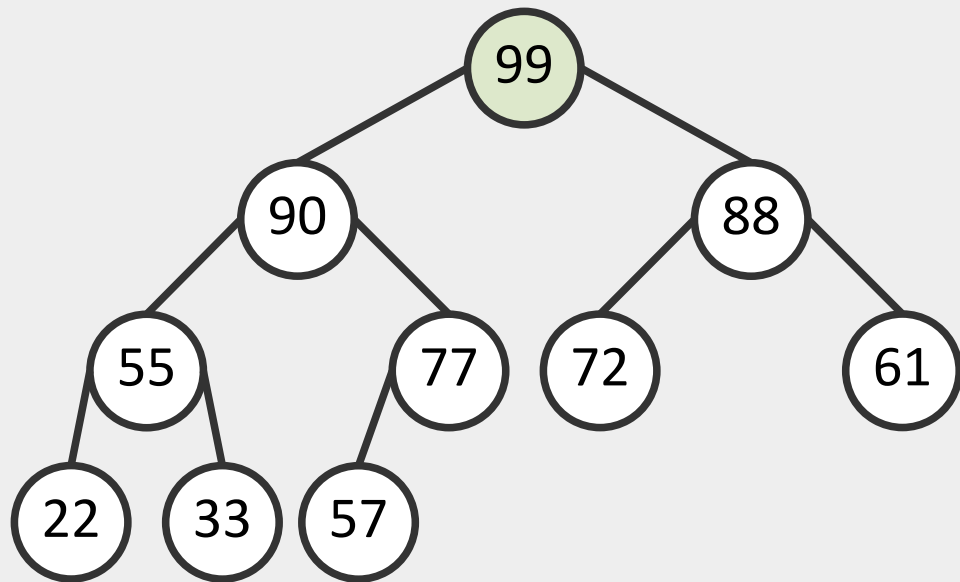
- Вставка элемента с приоритетом 90
- Добавляется на **текущий уровень** или на новый, если текущий заполнен
- **HeapifyUp** (двигаем элемент вверх по куче, если свойства нарушены)

Вставка элемента в бинарную кучу

```
int heap_insert(struct heap *h, int key, char *value)
{
    if (h->nnodes >= h->maxsize) /* Переполнение кучи */
        return -1;
    h->nnodes++;
    h->nodes[h->nnodes].key = key;
    h->nodes[h->nnodes].value = value;
    /* HeapifyUp */
    for (int i = h->nnodes;
         i > 1 && h->nodes[i].key > h->nodes[i / 2].key;
         i = i / 2)
        heap_swap(&h->nodes[i], &h->nodes[i / 2]);
    return 0;
}
```

$$T_{\text{Insert}} = O(\log n)$$

Удаление максимального элемента

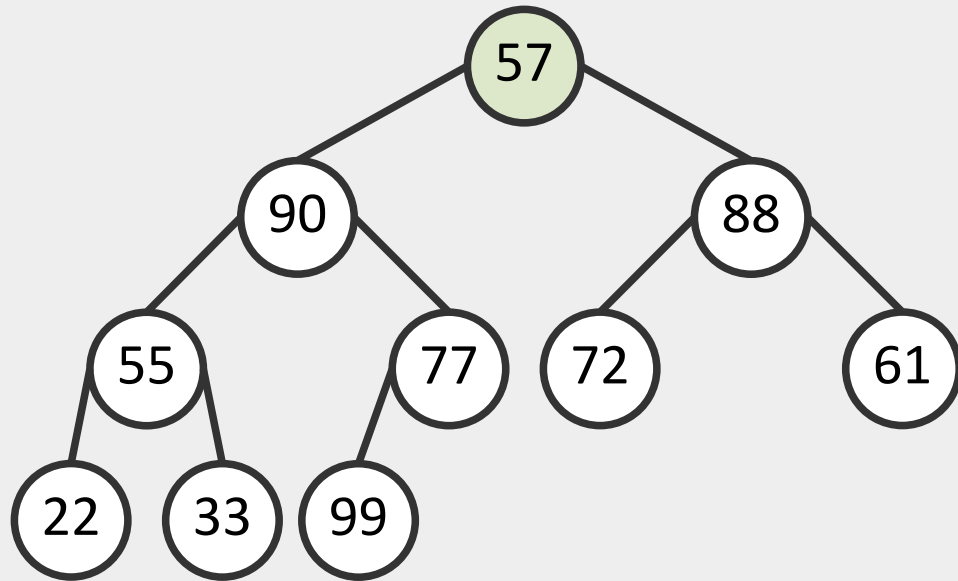


- Максимальный элемент в $H[0] = 99$

max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
99	90	88	55	77	72	61	22	33	57

Удаление максимального элемента

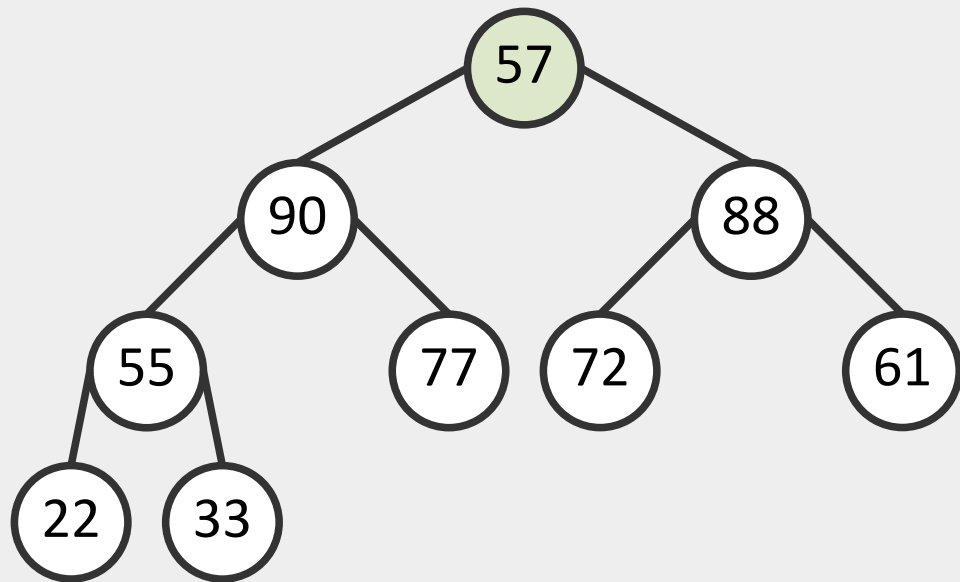


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
57	90	88	55	77	72	61	22	33	99

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)

Удаление максимального элемента

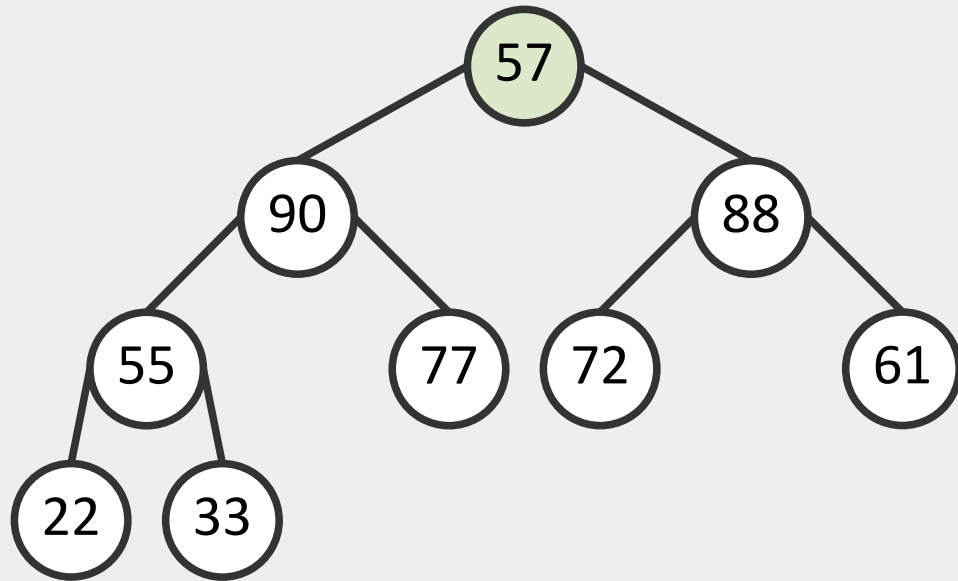


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
57	90	88	55	77	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел

Удаление максимального элемента

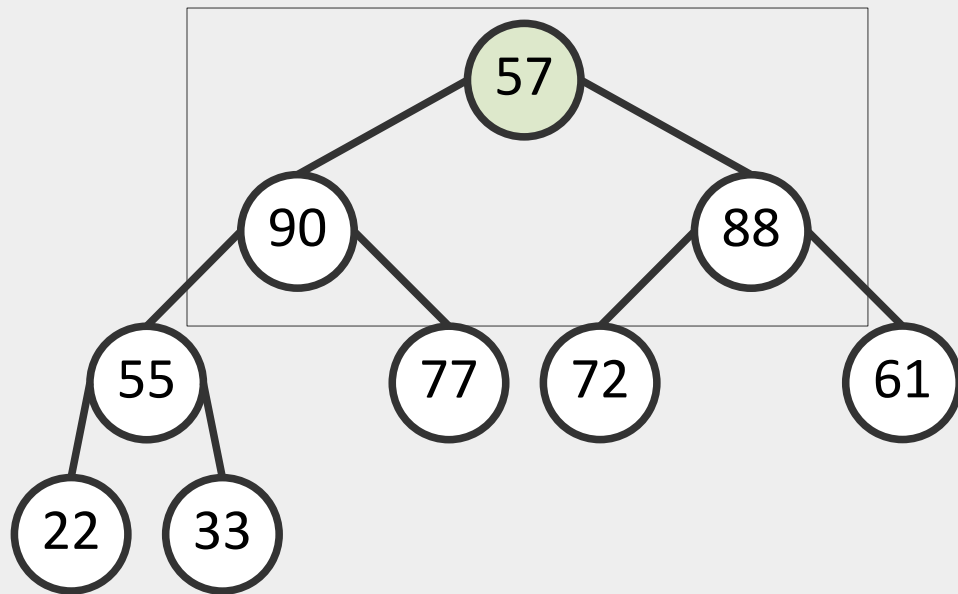


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
57	90	88	55	77	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел
- Восстанавливаем свойства кучи ($HeapifyDown(0)$)

Удаление максимального элемента

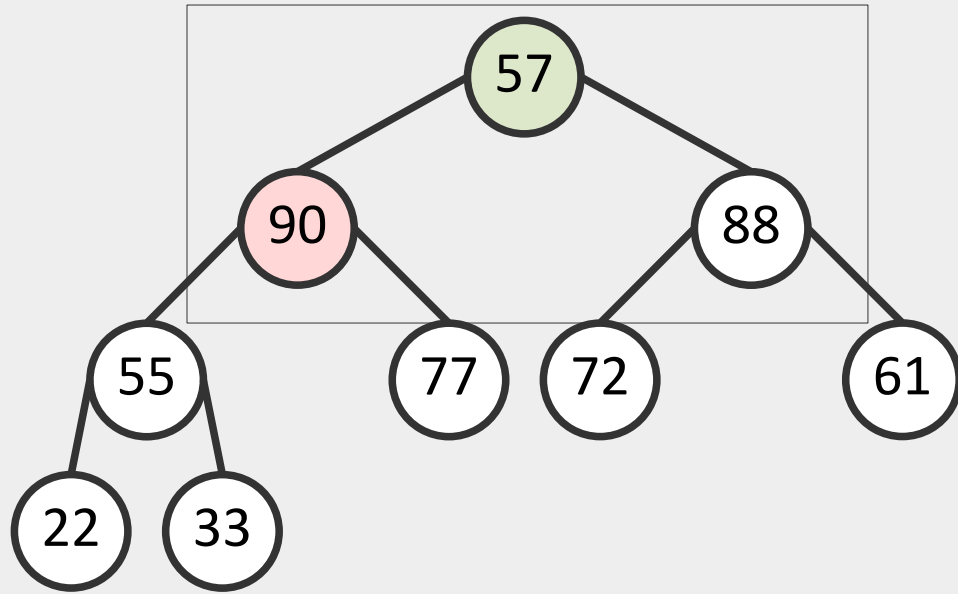


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
57	90	88	55	77	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел
- Восстанавливаем свойства кучи ($HeapifyDown(0)$)

Удаление максимального элемента

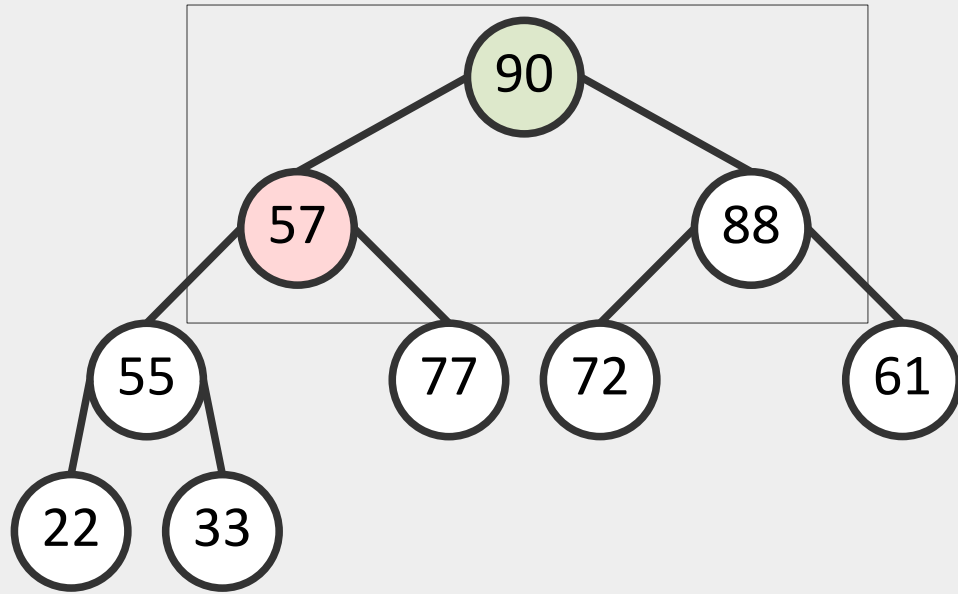


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
57	90	88	55	77	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел
- Восстанавливаем свойства кучи ($HeapifyDown(0)$)

Удаление максимального элемента

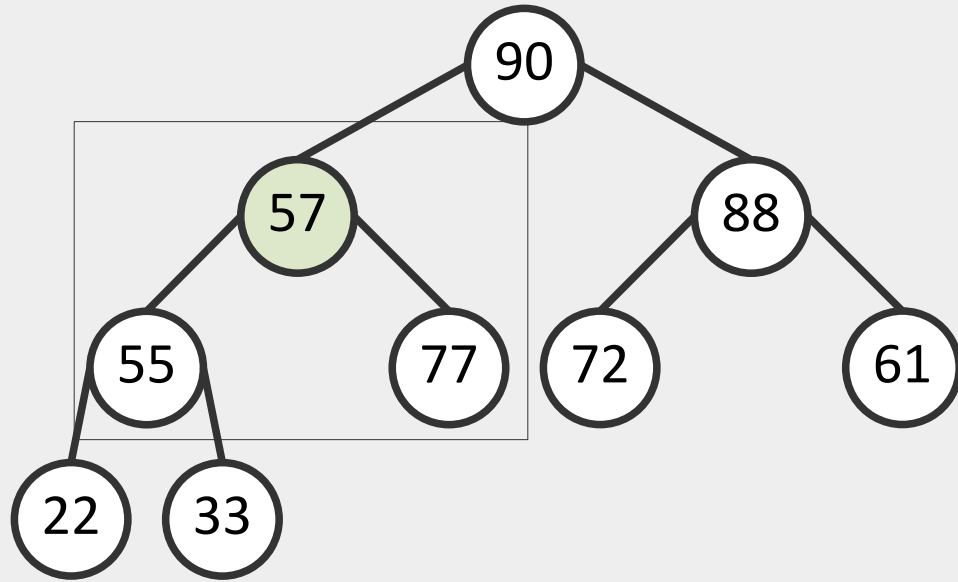


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
90	57	88	55	77	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел
- Восстанавливаем свойства кучи ($HeapifyDown(0)$)

Удаление максимального элемента

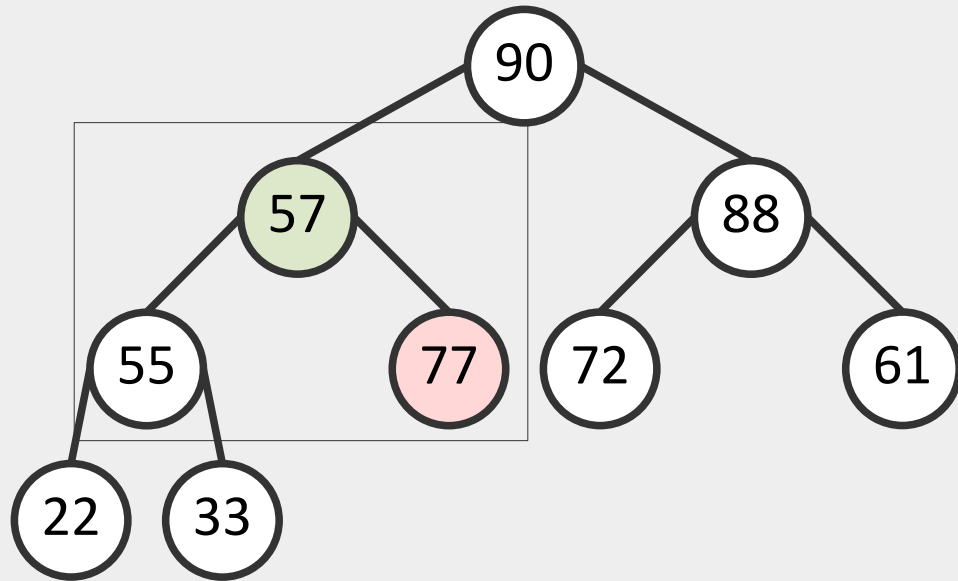


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
90	57	88	55	77	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел
- Восстанавливаем свойства кучи ($HeapifyDown(0)$)

Удаление максимального элемента

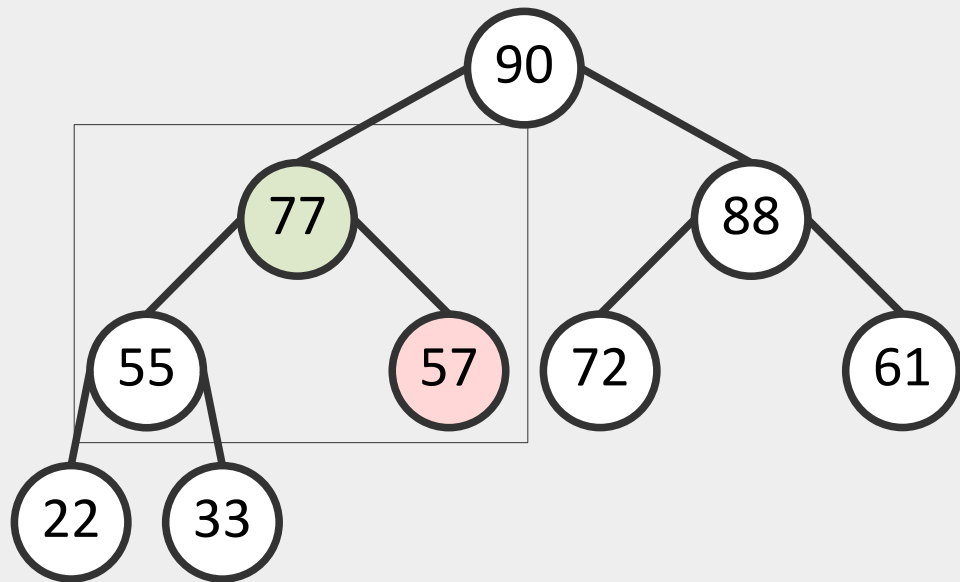


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
90	57	88	55	77	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел
- Восстанавливаем свойства кучи ($HeapifyDown(0)$)

Удаление максимального элемента

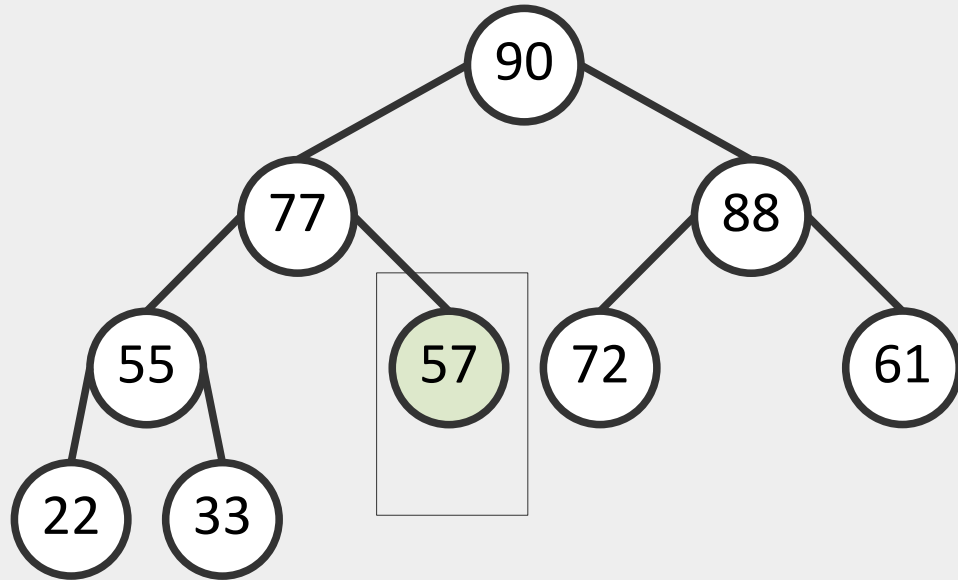


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
90	77	88	55	57	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел
- Восстанавливаем свойства кучи ($HeapifyDown(0)$)

Удаление максимального элемента

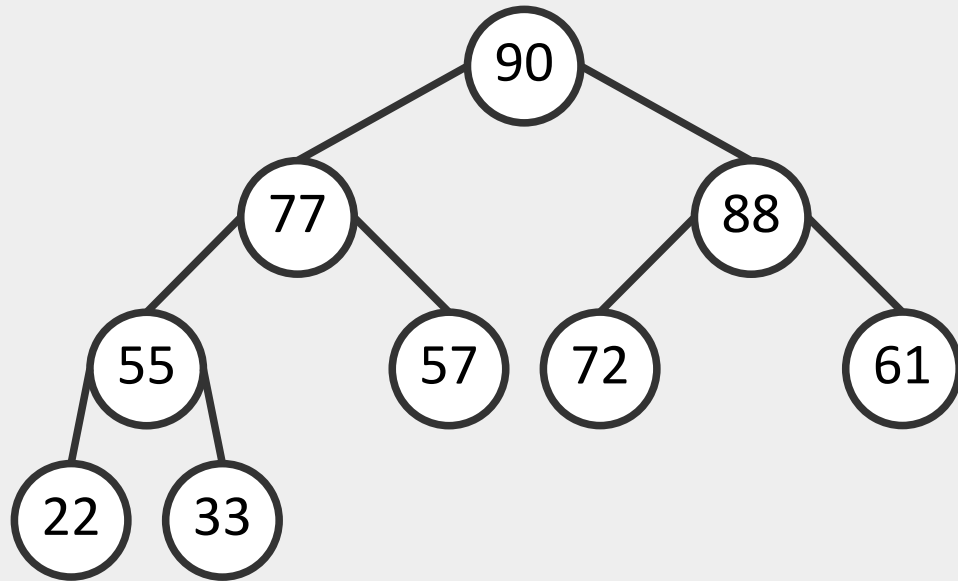


max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
90	77	88	55	57	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел
- Восстанавливаем свойства кучи ($HeapifyDown(0)$)

Удаление максимального элемента



max-heap, 10 элементов
Массив H приоритетов (ключей):

0	1	2	3	4	5	6	7	8	9
90	77	88	55	57	72	61	22	33	...

- Максимальный элемент в $H[0] = 99$
- Меняем местами с последним элементом в куче ($H[9]$)
- Удаляем последний узел
- Восстанавливаем свойства кучи ($HeapifyDown(0)$)

Удаление максимального элемента

```
struct heapnode heap_extract_max(struct heap *h)
{
    if (h->nnodes == 0)
        return (struct heapnode){0, NULL};
    struct heapnode maxnode = h->nodes[1];
    h->nodes[1] = h->nodes[h->nnodes--];
    heap_heapify(h, 1)
    return maxnode;
}
```

$$T_{\text{ExtractMax}} = O(\log n)$$

Восстановление свойств кучи

```
void heap_heapify(struct heap *h, int index)
{
    while (1) {
        int left = 2 * index;
        int right = 2 * index + 1;
        int largest = index;
        if (left <= h->nnodes &&
            h->nodes[left].key > h->nodes[largest].key)
            largest = left;
        if (right <= h->nnodes &&
            h->nodes[right].key > h->nodes[largest].key)
            largest = right;
        if (largest == index)
            break;
        heap_swap(&h->nodes[index], &h->nodes[largest]);
        index = largest;
    }
}
```

$$T_{\text{Heapify}} = O(\log n)$$

Увеличение приоритета элемента

```
int heap_increase_key(struct heap *h, int index, int newkey)
{
    if (h->nodes[index].key >= newkey)
        return -1;
    h->nodes[index].key = newkey;
    while (index > 1 &&
           h->nodes[index].key > h->nodes[index / 2].key) {
        heap_swap(&h->nodes[index], &h->nodes[index / 2]);
        index /= 2;
    }
    return index;
}
```

$$T_{\text{IncreaseKey}} = O(\log n)$$

Построение бинарной кучи

- Дан неупорядоченный массив A длины n
- Требуется построить из его элементов бинарную кучу

```
void BuildHeap(int *A, int n)
{
    struct heap *h = heap_create(n);           // 0(1)
    for (int i = 0; i < n; i++) {               // for:  $n$ 
        heap_insert(h, A[i]);                   // 0(log $n$ )
    }
}
```

$$T_{\text{BuildHeap}} = O(n \log n)$$

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины n
- Требуется построить из его элементов бинарную кучу

```
void *build_max_heap(int *arr, int n)
{
    for (int i = floor(n / 2) - 1; i > -1; i--) {
        heapify_down(arr, n, i);
    }
}
```

$$T_{\text{BuildHeap}} = O(n)$$

Построение бинарной кучи за $O(n)$

```
void heapify_down(int *arr, int n, int i)
{
    while (i < n) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;
        if (left < n && arr[left] > arr[largest]) {
            largest = left;
        }
        if (right < n && arr[right] > arr[largest]) {
            largest = right;
        }
        if (largest != i) {
            swap(arr[i], arr[largest]);
            i = largest;
        } else { break; }
    }
}
```

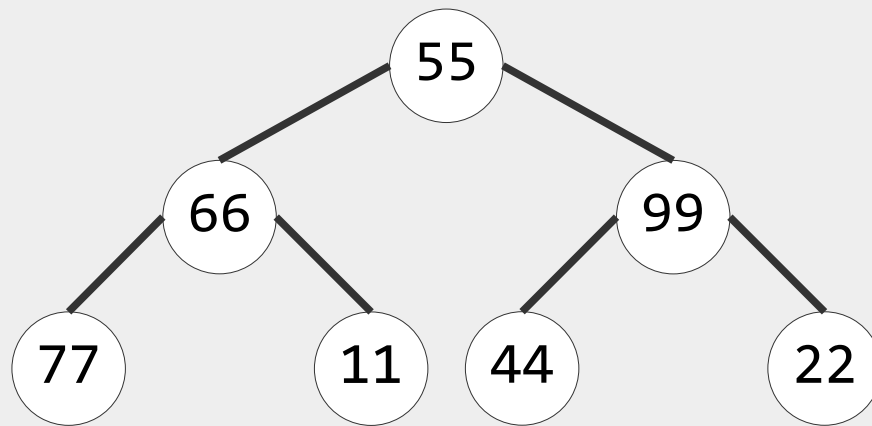
$$T_{\text{HeapifyDown}} = O(\log n)$$

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



0	1	2	3	4	5	6
55	66	99	77	11	44	22

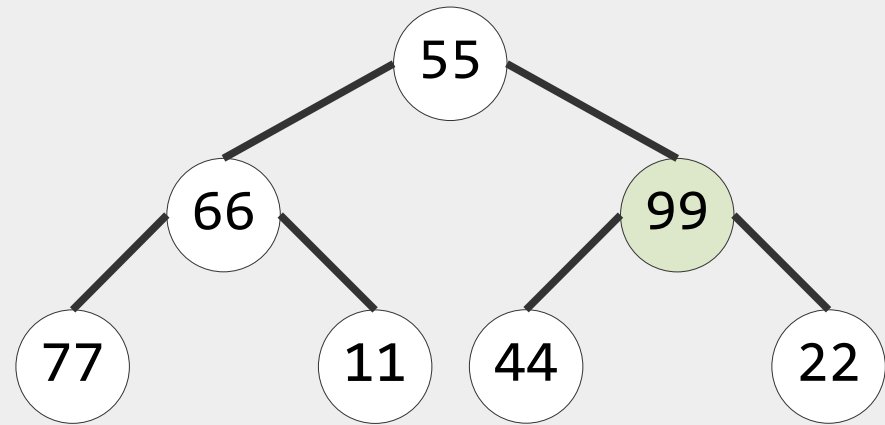
Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу

$$i = 7 / 2 - 1 = 3 - 1 = 2$$

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



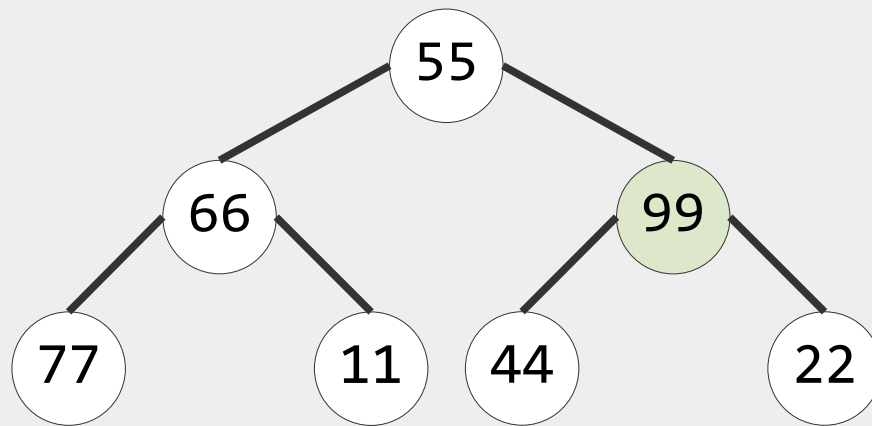
0	1	2	3	4	5	6
55	66	99	77	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
`heapify_down(arr, 7, 2)`

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



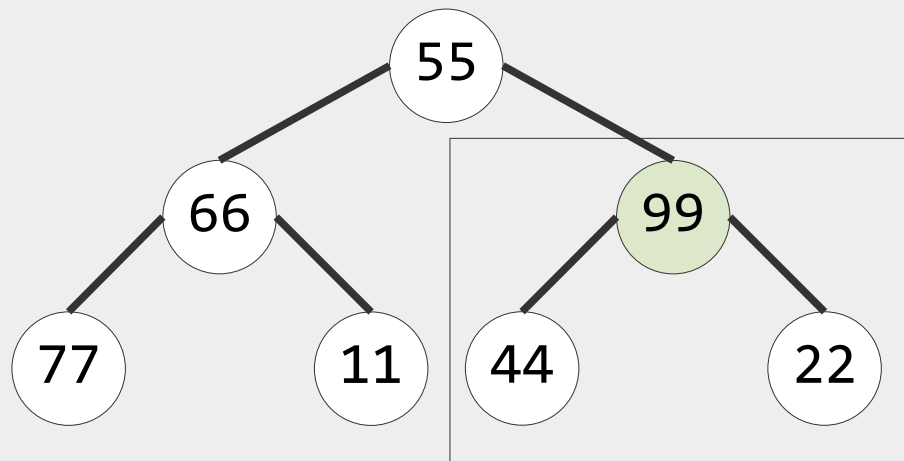
0	1	2	3	4	5	6
55	66	99	77	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
`heapify_down(arr, 7, 2)`

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



0	1	2	3	4	5	6
55	66	99	77	11	44	22

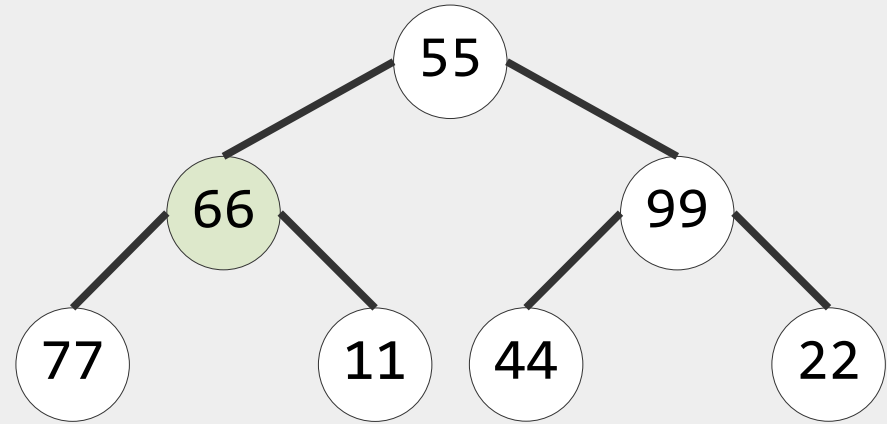
Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу

$$i = 2 - 1 = 1$$

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



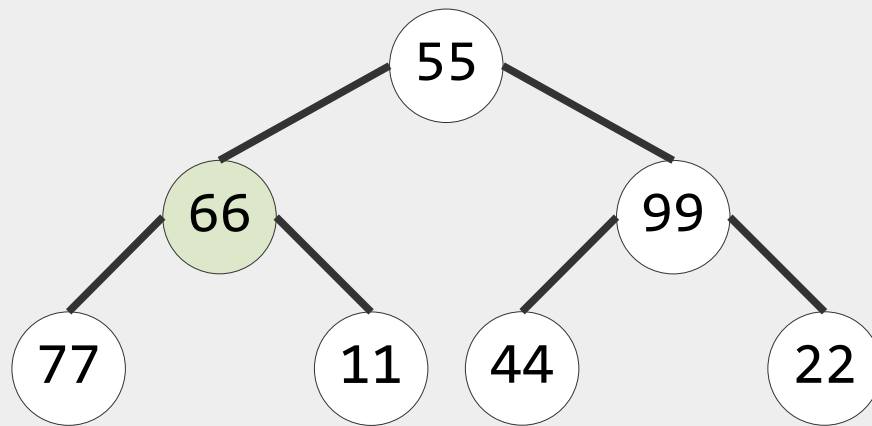
0	1	2	3	4	5	6
55	66	99	77	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
`heapify_down(arr, 7, 1)`

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



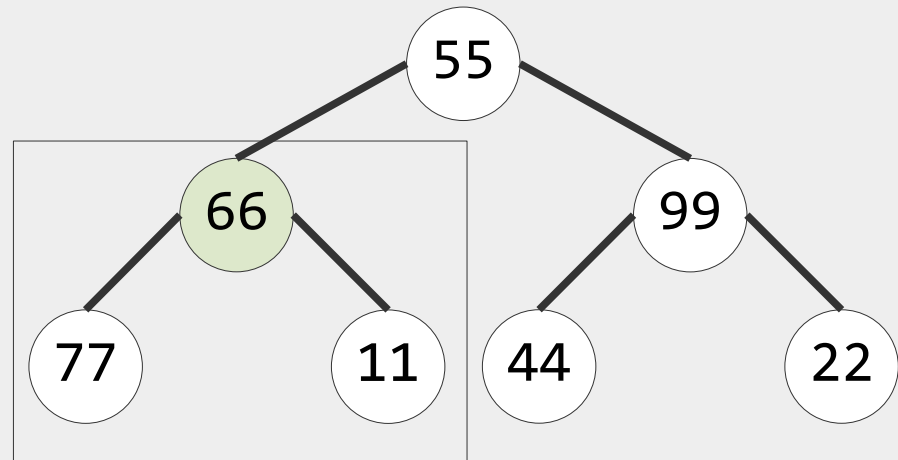
0	1	2	3	4	5	6
55	66	99	77	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
`heapify_down(arr, 7, 1)`

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



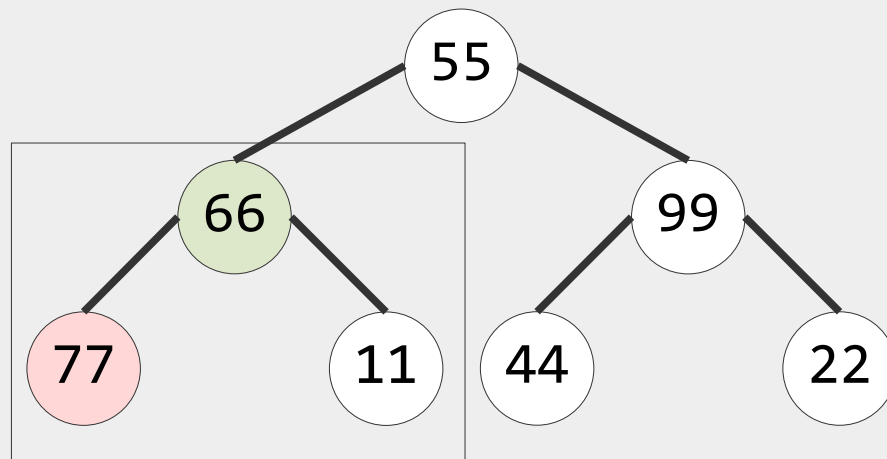
0	1	2	3	4	5	6
55	66	99	77	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
`heapify_down(arr, 7, 1)`

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



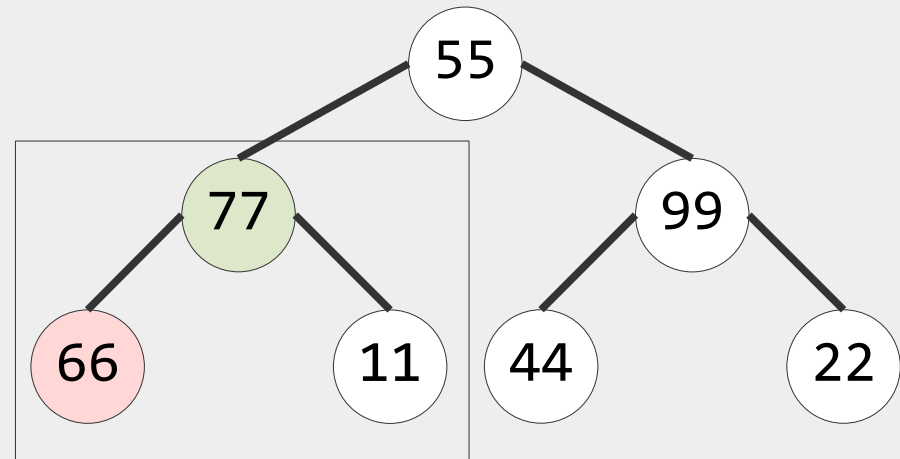
0	1	2	3	4	5	6
55	66	99	77	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
`heapify_down(arr, 7, 1)`

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



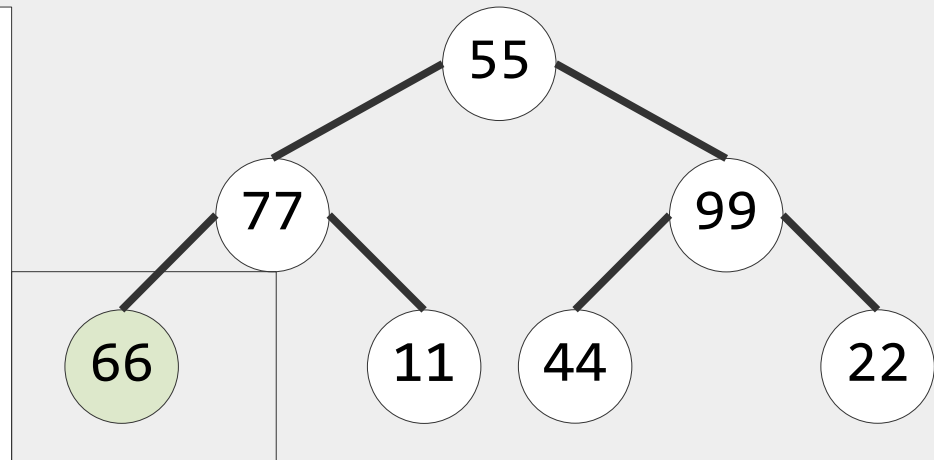
0	1	2	3	4	5	6
55	77	99	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
`heapify_down(arr, 7, 1)`

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



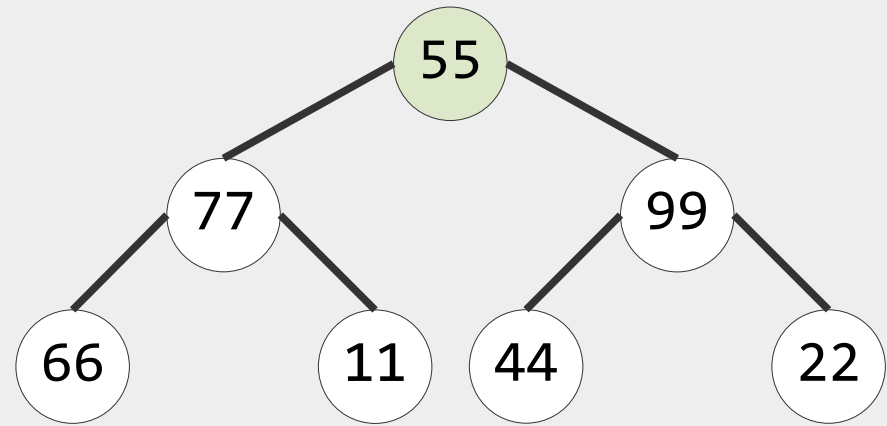
0	1	2	3	4	5	6
55	77	99	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
 $i = 2 - 1 - 1 = 0$

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



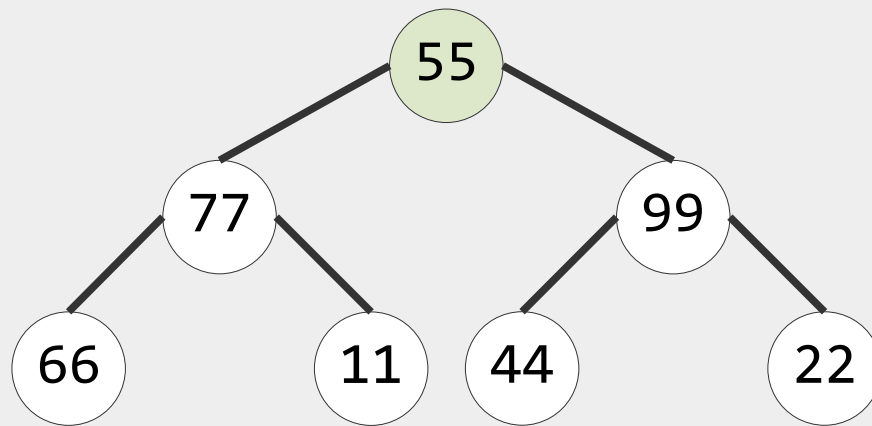
0	1	2	3	4	5	6
55	77	99	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
 $\text{heapify_down}(\text{arr}, 7, 0)$

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



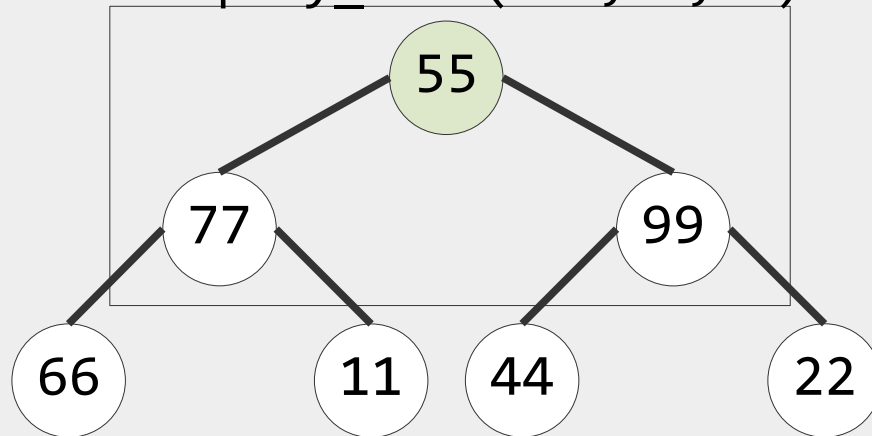
0	1	2	3	4	5	6
55	77	99	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
`heapify_down(arr, 7, 0)`

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



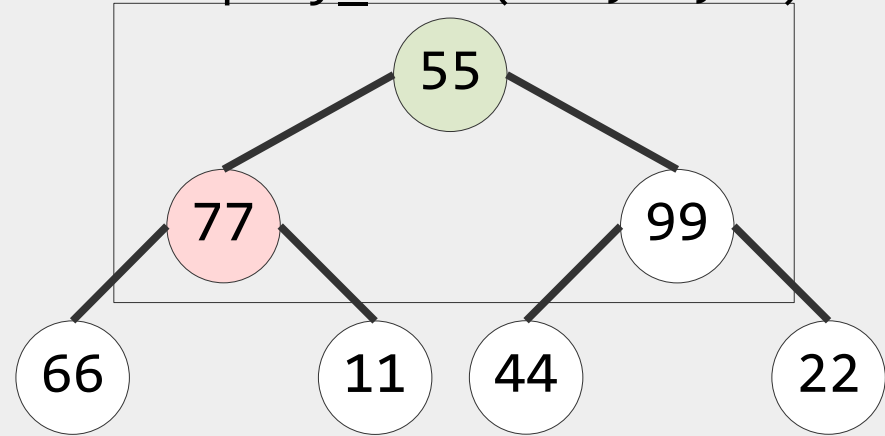
0	1	2	3	4	5	6
55	77	99	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
 $\text{heapify_down}(\text{arr}, 7, 0)$

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



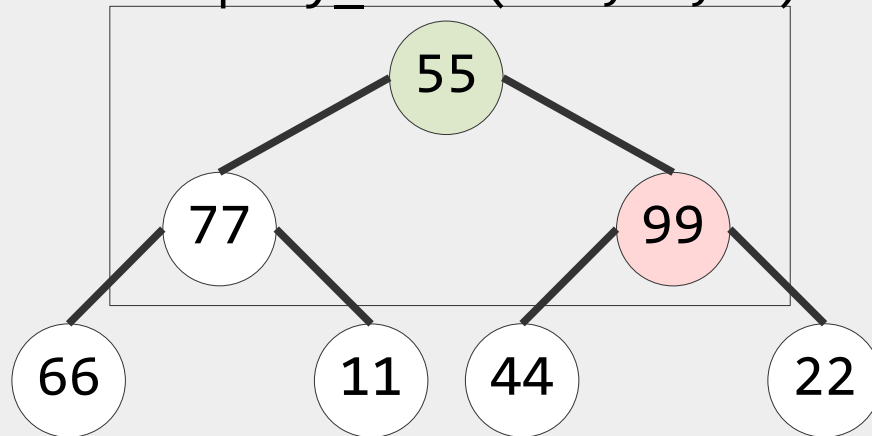
0	1	2	3	4	5	6
55	77	99	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
 $\text{heapify_down}(\text{arr}, 7, 0)$

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



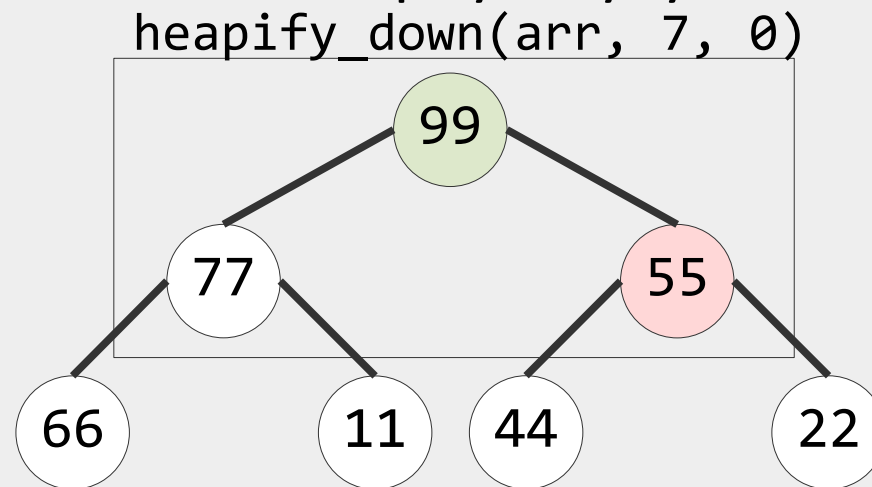
0	1	2	3	4	5	6
55	77	99	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
         i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



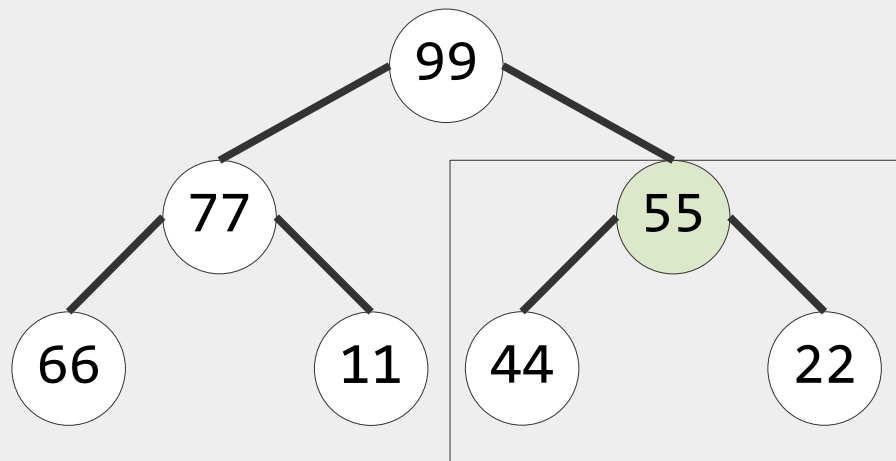
0	1	2	3	4	5	6
99	77	55	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу
 $\text{heapify_down}(\text{arr}, 7, 0)$

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



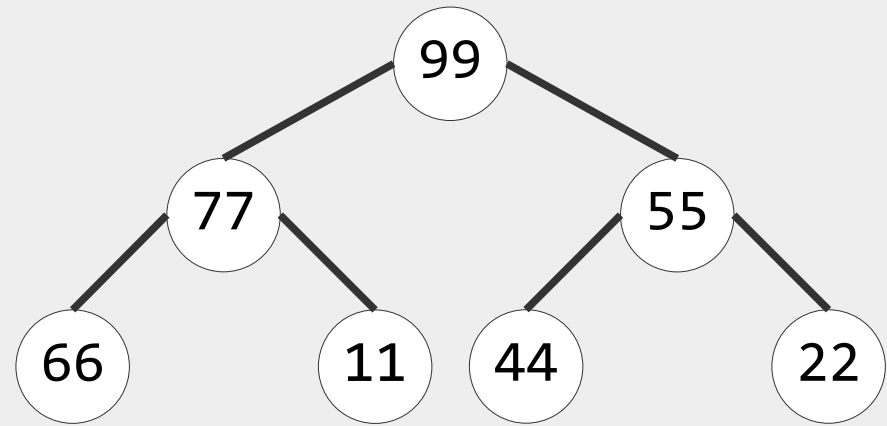
0	1	2	3	4	5	6
99	77	55	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу

```
void *build_max_heap(int *arr,  
                    int n)  
{  
    for (int i = floor(n / 2) - 1;  
        i > -1; i--) {  
        heapify_down(arr, n, i);  
    }  
}
```

$$T_{\text{BuildHeap}} = O(n)$$



0	1	2	3	4	5	6
99	77	55	66	11	44	22

Построение бинарной кучи за $O(n)$

- Дан неупорядоченный массив A длины $n = 7$
- Требуется построить из его элементов бинарную кучу

```
void *build_max_heap(int *arr,
                    int n)
{
    for (int i = floor(n / 2) - 1;
        i > -1; i--) {
        heapify_down(arr, n, i);
    }
}
```

$$T_{\text{BuildHeap}} = O(n)$$

$$T(n) = \sum_{h=1}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=1}^{\lfloor \log_2 n \rfloor} \frac{h}{2^h}\right)$$
$$\sum_{h=1}^{\lfloor \log_2 n \rfloor} h \left(\frac{1}{2}\right)^h < \sum_{h=0}^{\infty} h \left(\frac{1}{2}\right)^h = \frac{1/2}{(1-1/2)^2} = 2$$
$$T(n) = O\left(n \sum_{h=1}^{\lfloor \log_2 n \rfloor} \frac{h}{2^h}\right) = O(n)$$

0	1	2	3	4	5	6
99	77	55	66	11	44	22

Работа с бинарной кучей

```
int main()
{
    struct heap *h;
    struct heapnode node;
    h = heap_create(100);
    heap_insert(h, 33, "33");
    heap_insert(h, 10, "10");
    heap_insert(h, 21, "21");
    heap_insert(h, 17, "17");
    heap_insert(h, 12, "12");
    heap_insert(h, 55, "55");
    heap_insert(h, 22, "22");
    heap_insert(h, 44, "44");
    heap_insert(h, 25, "25");
    node = heap_extract_max(h);
    printf("Max: %d\n", node.key);
    int i = heap_increase_key(h, 22, 100);
    heap_free(h);
    return 0;
}
```

Сортировка на основе бинарной кучи (Heap Sort)

- На основе бинарной кучи можно реализовать алгоритм сортировки с вычислительной сложностью $O(n \log n)$ в худшем случае

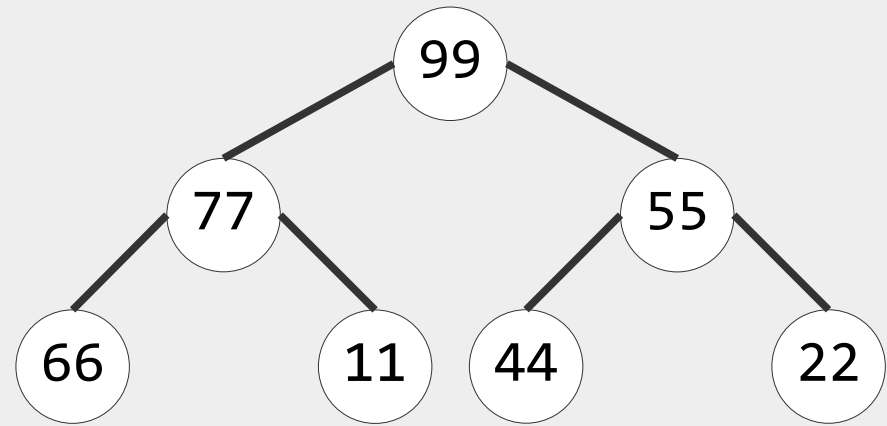
```
void *heap_sort(int *arr, int n)
{
    build_max_heap(arr, n);           // n
    for (int i = n - 1; i > 0; i--) { // n
        swap(arr[0], arr[i]);
        heapify_down(arr, i, 0);      // log n
    }
}
```

$$T_{\text{HeapSort}} = O(n \log n)$$

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 6$
- `swap(A[0], A[i])`
- `heapify(A, i, 0)`

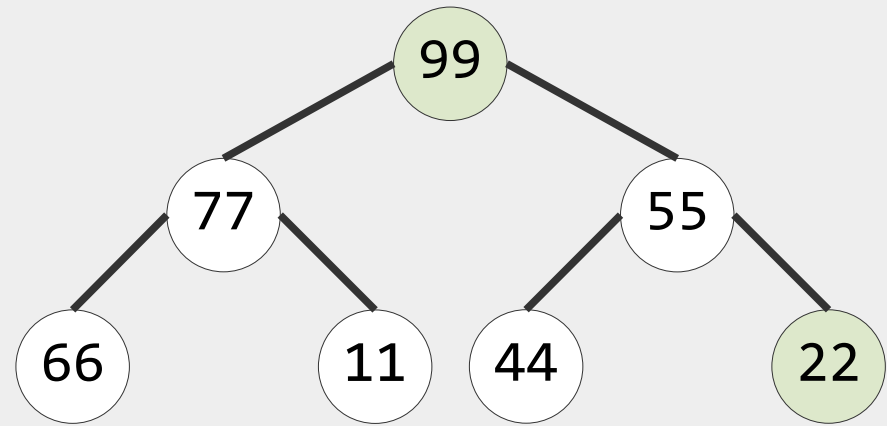


0	1	2	3	4	5	6
99	77	55	66	11	44	22

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 6$
- `swap(A[0], A[6])`
- `heapify(A, 6, 0)`

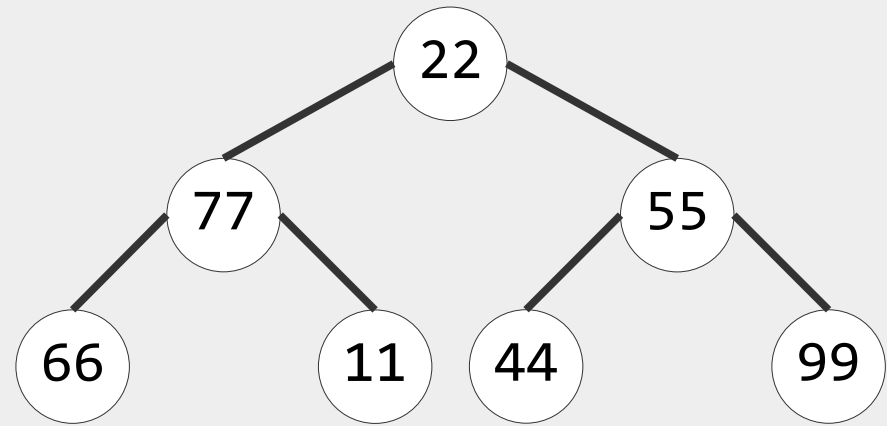


0	1	2	3	4	5	6
99	77	55	66	11	44	22

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 6$
- `swap(A[0], A[6])`
- `heapify(A, 6, 0)`

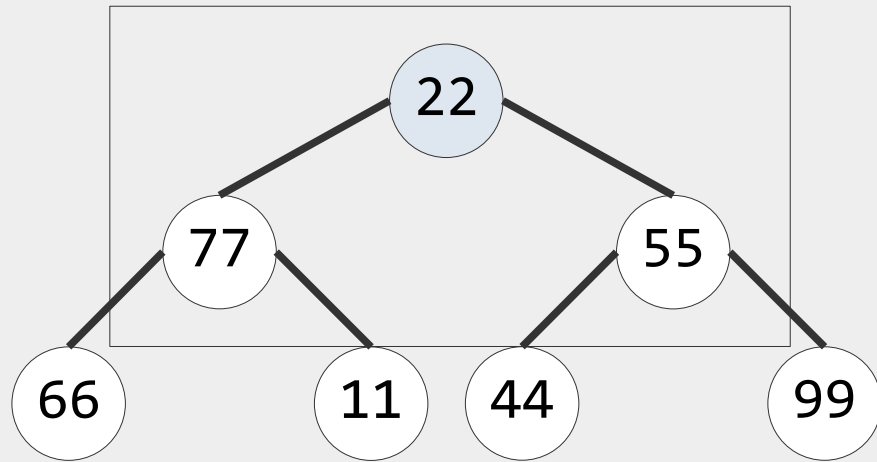


0	1	2	3	4	5	6
22	77	55	66	11	44	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 6$
- `swap(A[0], A[6])`
- `heapify(A, 6, 0)`

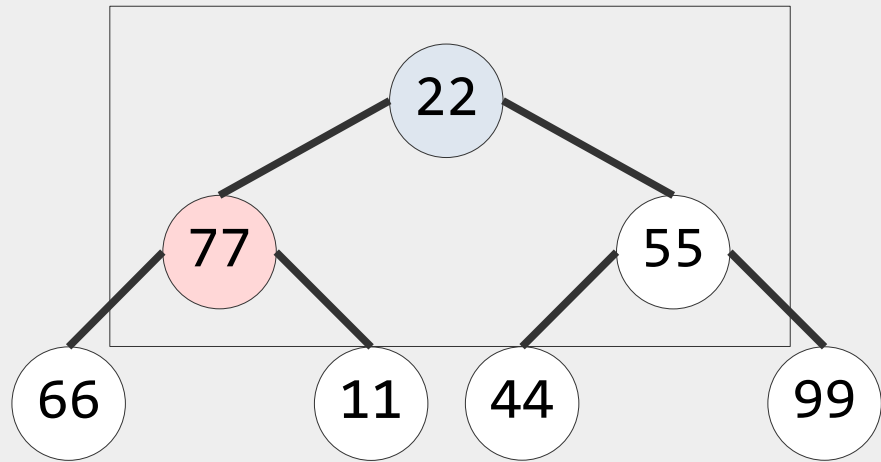


0	1	2	3	4	5	6
22	77	55	66	11	44	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 6$
- `swap(A[0], A[6])`
- `heapify(A, 6, 0)`

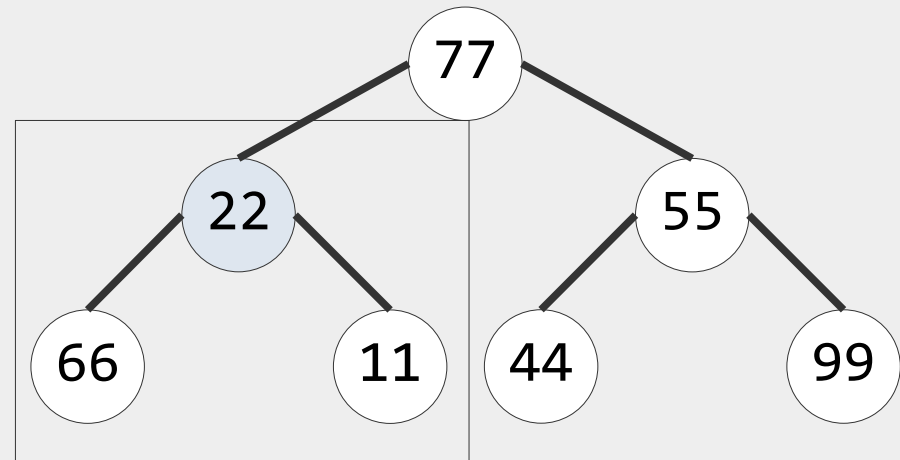


0	1	2	3	4	5	6
22	77	55	66	11	44	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 6$
- `swap(A[0], A[6])`
- `heapify(A, 6, 0)`

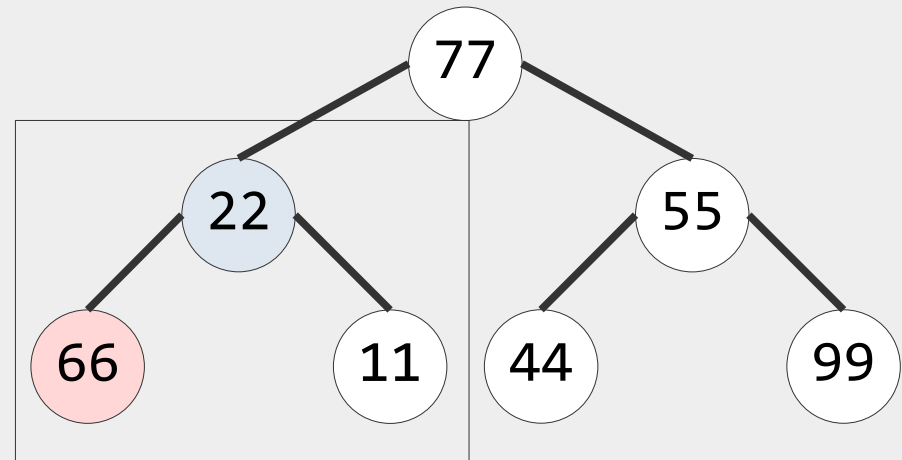


0	1	2	3	4	5	6
77	22	55	66	11	44	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 6$
- `swap(A[0], A[6])`
- `heapify(A, 6, 0)`

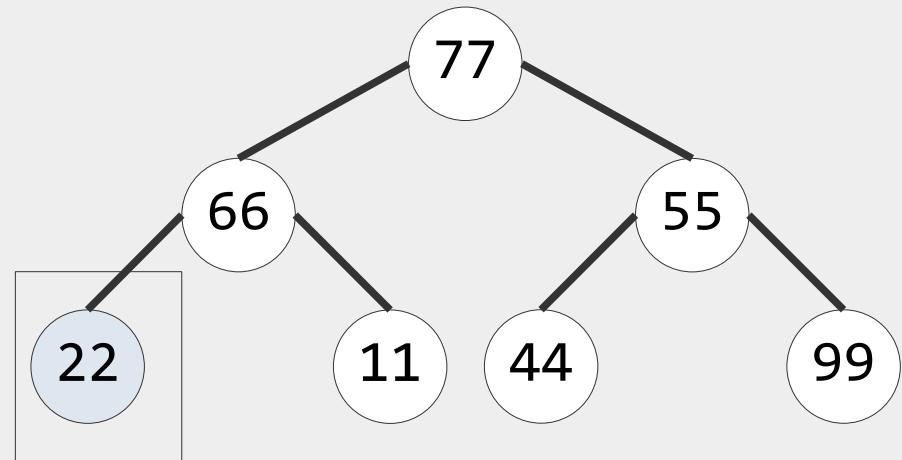


0	1	2	3	4	5	6
77	22	55	66	11	44	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 6$
- `swap(A[0], A[6])`
- `heapify(A, 6, 0)`

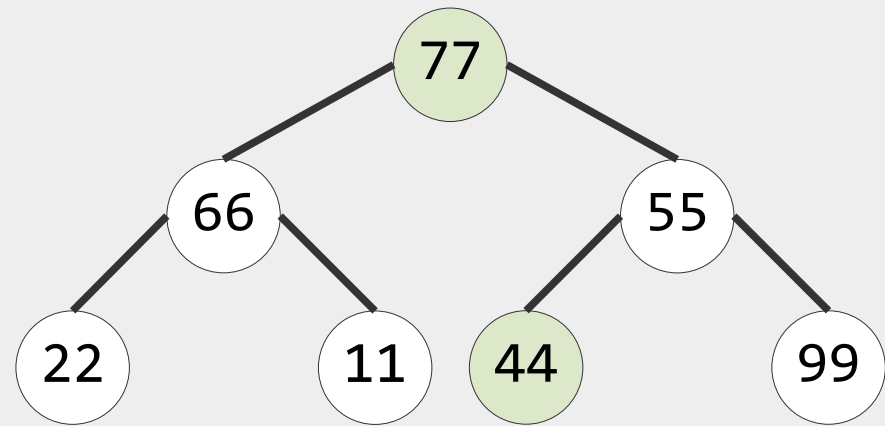


0	1	2	3	4	5	6
77	66	55	22	11	44	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 5$
- `swap(A[0], A[5])`
- `heapify(A, 5, 0)`

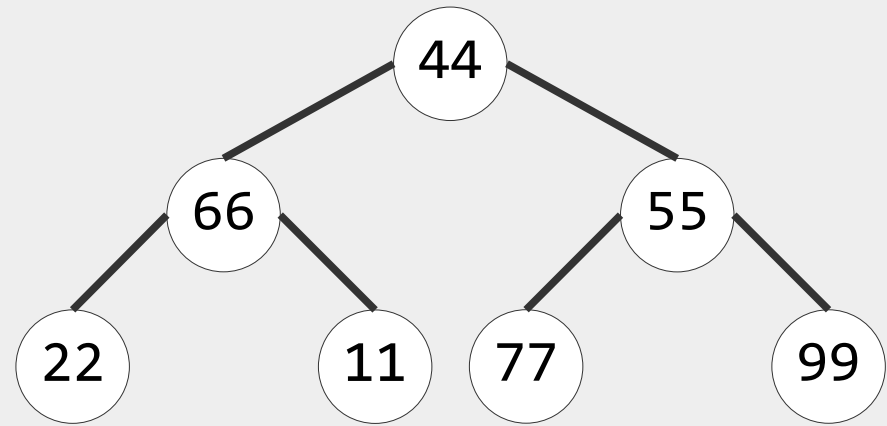


0	1	2	3	4	5	6
77	66	55	22	11	44	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 5$
- `swap(A[0], A[5])`
- `heapify(A, 5, 0)`

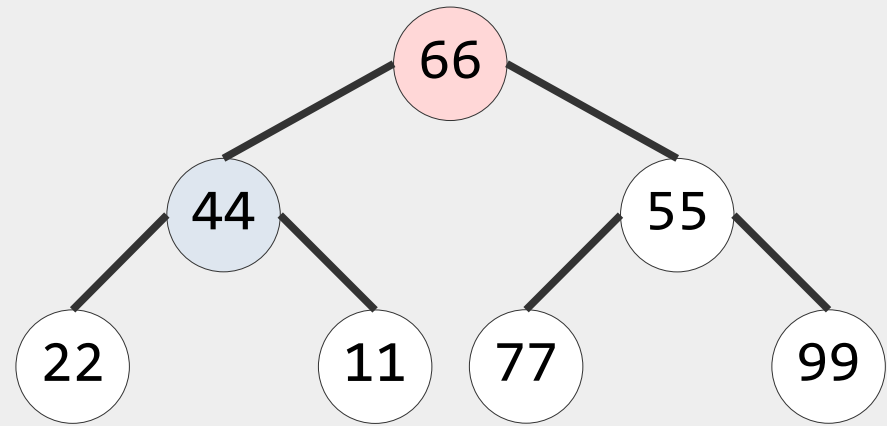


0	1	2	3	4	5	6
44	66	55	22	11	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 5$
- `swap(A[0], A[5])`
- `heapify(A, 5, 0)`

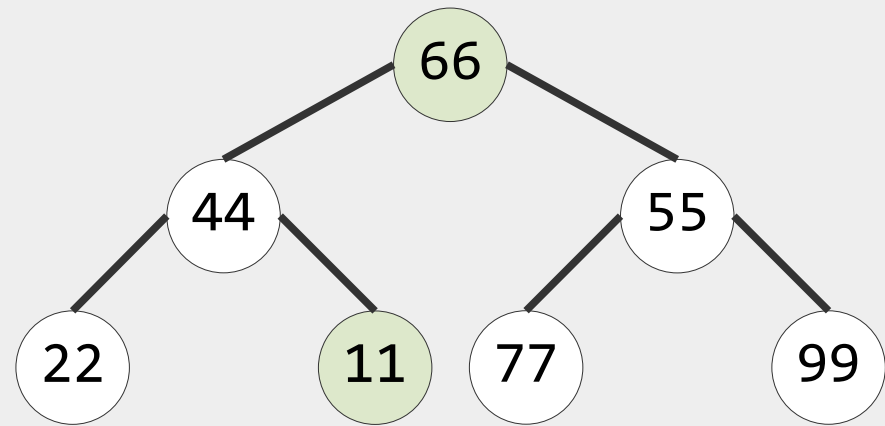


0	1	2	3	4	5	6
66	44	55	22	11	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 4$
- `swap(A[0], A[4])`
- `heapify(A, 4, 0)`

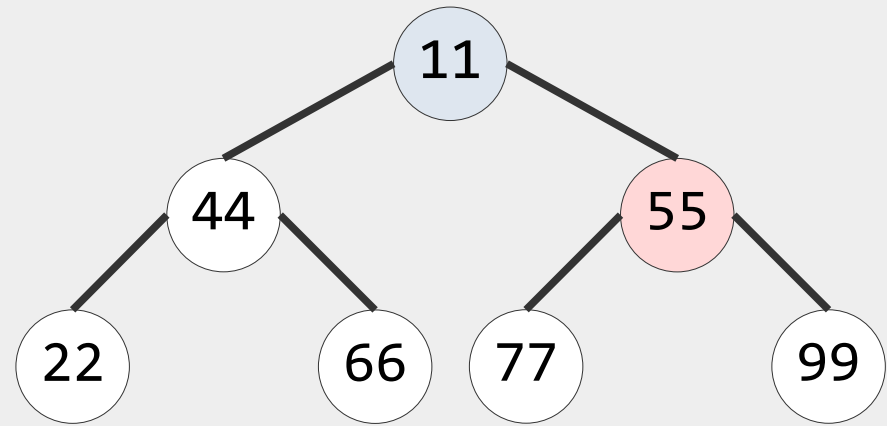


0	1	2	3	4	5	6
66	44	55	22	11	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 4$
- `swap(A[0], A[4])`
- `heapify(A, 4, 0)`

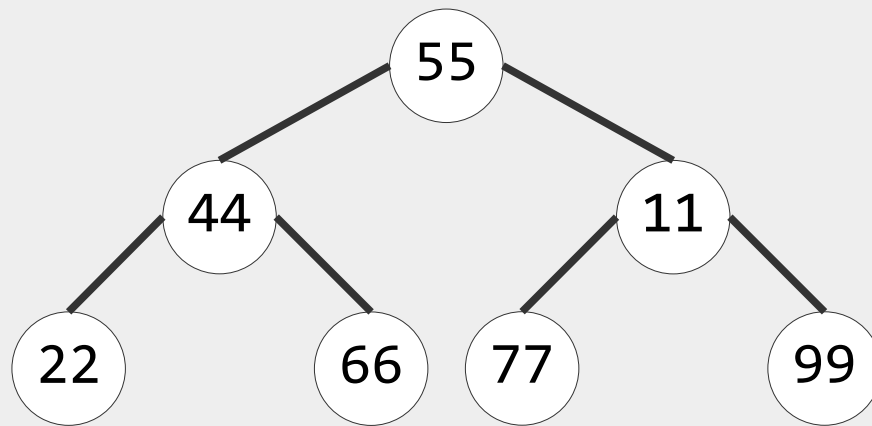


0	1	2	3	4	5	6
11	44	55	22	66	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 4$
- `swap(A[0], A[4])`
- `heapify(A, 4, 0)`

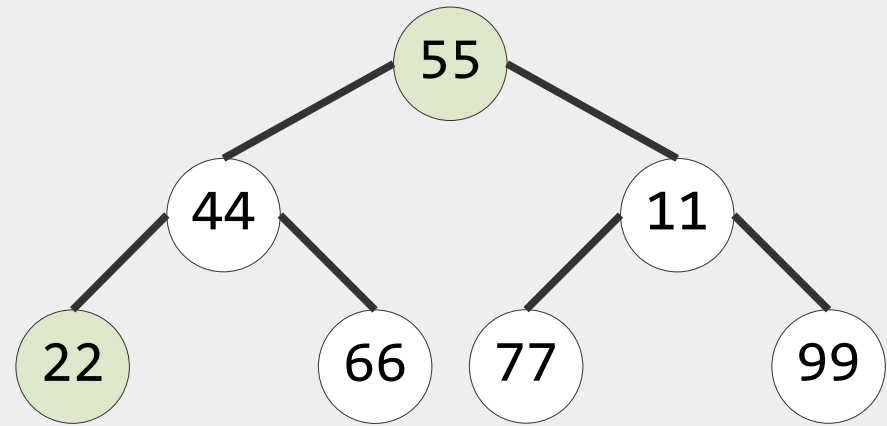


0	1	2	3	4	5	6
55	44	11	22	66	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 3$
- `swap(A[0], A[3])`
- `heapify(A, 3, 0)`

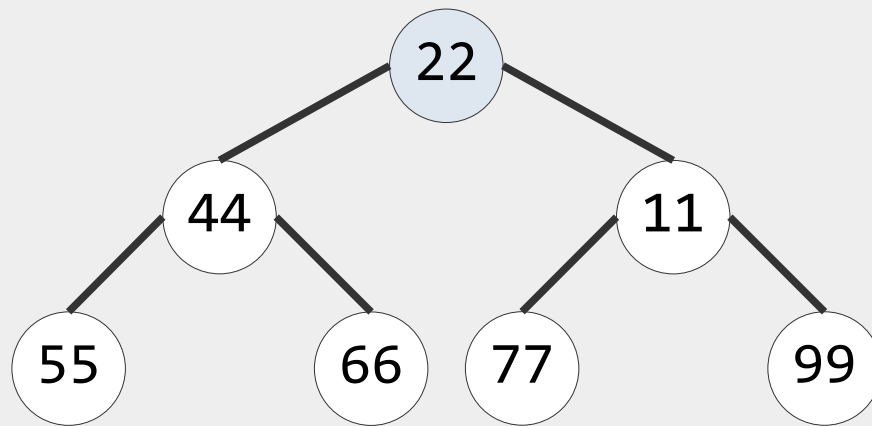


0	1	2	3	4	5	6
55	44	11	22	66	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 3$
- `swap(A[0], A[3])`
- `heapify(A, 3, 0)`

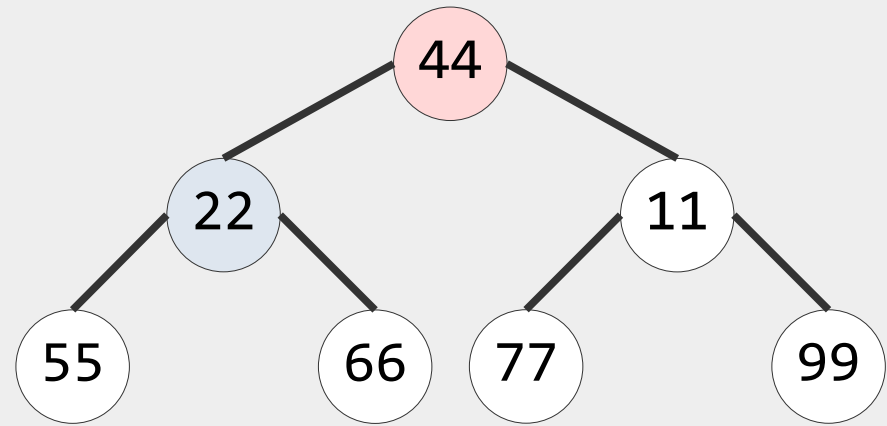


0	1	2	3	4	5	6
22	44	11	55	66	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 3$
- `swap(A[0], A[3])`
- `heapify(A, 3, 0)`

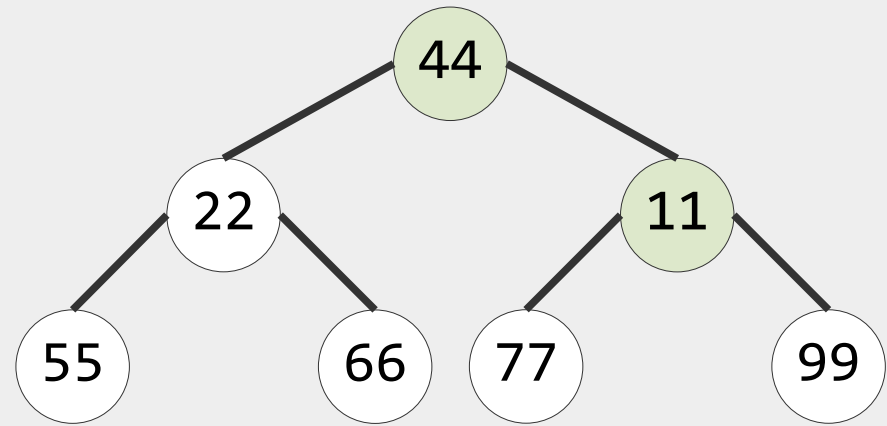


0	1	2	3	4	5	6
44	22	11	55	66	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 2$
- `swap(A[0], A[2])`
- `heapify(A, 2, 0)`

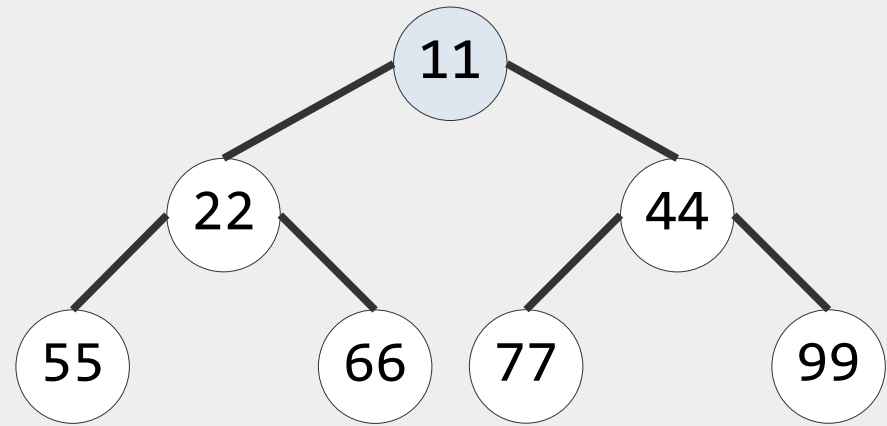


0	1	2	3	4	5	6
44	22	11	55	66	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 2$
- `swap(A[0], A[2])`
- `heapify(A, 2, 0)`

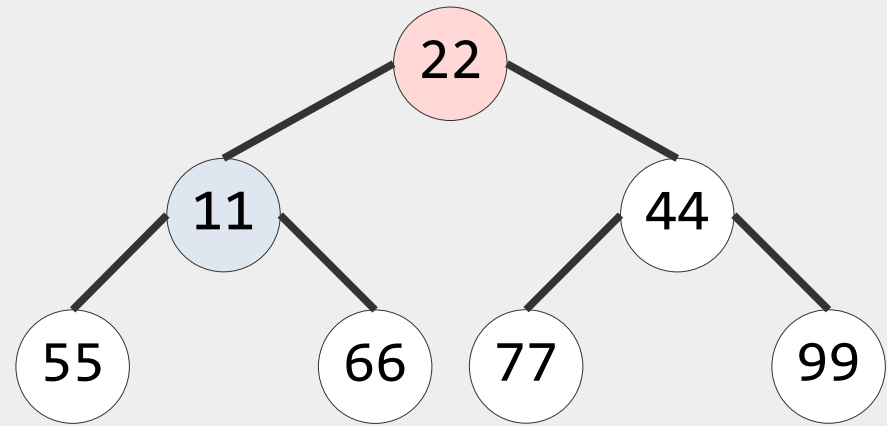


0	1	2	3	4	5	6
11	22	44	55	66	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 2$
- `swap(A[0], A[2])`
- `heapify(A, 2, 0)`

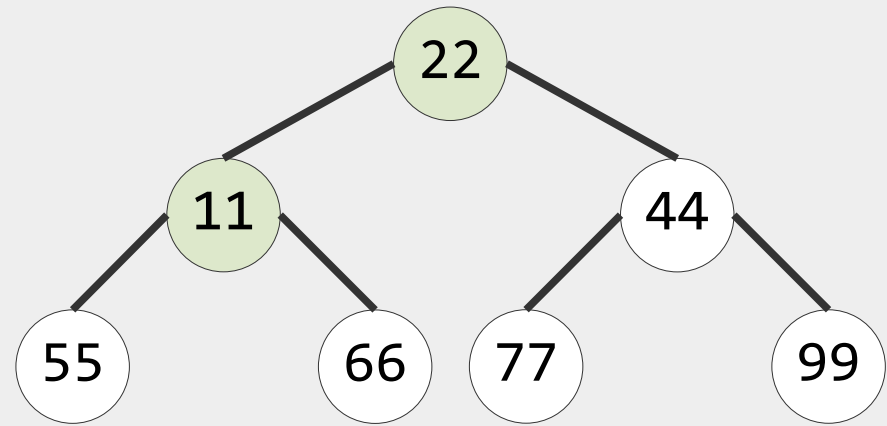


0	1	2	3	4	5	6
22	11	44	55	66	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 1$
- `swap(A[0], A[1])`
- `heapify(A, 1, 0)`

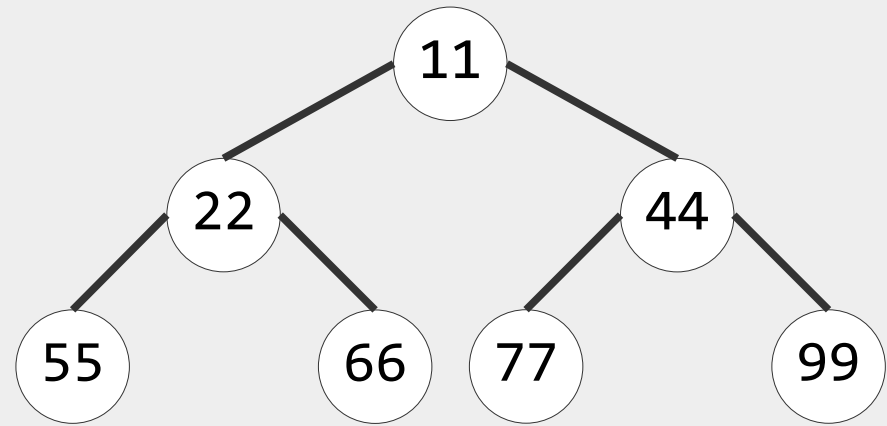


0	1	2	3	4	5	6
22	11	44	55	66	77	99

Сортировка на основе бинарной кучи (Heap Sort)

- Дан неупорядоченный массив A длины $n = 7$
- Ранее выполнено `build_max_heap`

- $i = 1$
- `swap(A[0], A[1])`
- `heapify(A, 1, 0)`



0	1	2	3	4	5	6
11	22	44	55	66	77	99

Очередь с приоритетом (priority queue)

- В таблице приведены трудоемкости операций очереди с приоритетом (в худшем случае, worst case)
- Символом «*» отмечена амортизированная сложность операций

Операция	Binary heap	Binomial heap	Fibonacci heap	Pairing heap	Brodal heap
FindMin	$\Theta(1)$	$O(\log n)$	$\Theta(1)$	$\Theta(1)^*$	$\Theta(1)$
DeleteMin	$\Theta(\log n)$	$\Theta(\log n)$	$O(\log n)^*$	$O(\log n)^*$	$O(\log n)$
Insert	$\Theta(\log n)$	$O(\log n)$	$\Theta(1)$	$\Theta(1)^*$	$\Theta(1)$
DecreaseKey	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)^*$	$O(\log n)^*$	$\Theta(1)$
Merge/Union	$\Theta(n)$	$\Omega(\log n)$	$\Theta(1)$	$\Theta(1)^*$	$\Theta(1)$

Бинарная куча в ядре Linux

```
/* nr - количество элементов в куче на данный момент
   size - максимальное количество элементов в куче
   *data - указатель на начало массива, содержащего элементы
   preallocated - начало статического массива,
                  содержащего элементы кучи */
#define MIN_HEAP_PREALLOCATED(_type, _name, _nr) \
struct _name { \
    size_t nr; \
    size_t size; \
    _type *data; \
    _type preallocated[_nr]; \
}
```

Бинарная куча в ядре Linux

```
/* less - функция сравнения, swp - функция для обмена элементов */
struct min_heap_callbacks {
    bool (*less)(const void *lhs, const void *rhs, void *args);
    void (*swp)(void *lhs, void *rhs, void *args);
};

/* i - индекс элемента
   lbist - предварительно вычисленная
          1-битная маска = "size & -size"
   size - размер каждого элемента
*/

static size_t parent(size_t i, unsigned int lsbit, size_t size)
{
    i -= size;
    i -= size & -(i & lsbit);
    return i / 2;
}
```

Бинарная куча в ядре Linux, добавление

```
bool __min_heap_push_inline(min_heap_char *heap, const void *element,
    size_t elem_size, const struct min_heap_callbacks *func, void *args)
{
    void *data = heap->data;
    size_t pos;
    if (heap->nr >= heap->size) /* Если куча заполнена */
        return false;

    /* Добавление в конец кучи */
    pos = heap->nr;
    memcpy(data + (pos * elem_size), element, elem_size);
    heap->nr++;

    /* Сдвиг дочернего элемента вверх */
    __min_heap_sift_up_inline(heap, elem_size, pos, func, args);
    return true;
}
```

$$T_{\text{Insert}} = O(\log n)$$

Бинарная куча в ядре Linux, добавление

```
void __min_heap_sift_up_inline(min_heap_char *heap, size_t elem_size,
                              size_t idx, const struct min_heap_callbacks *func, void *args)
{
    const unsigned long lsbite = elem_size & -elem_size;
    void *data = heap->data;
    void (*swp)(void *lhs, void *rhs, void *args) = func->swp;
    /* Предварительно масштабировать счетчики для производительности */
    size_t a = idx * elem_size, b;
    /* Выбор функции обмена */
    if (!swp)
        swp = select_swap_func(data, elem_size);
    /* Цикл обмена элементов, пока не break */
    while (a) {
        b = parent(a, lsbite, elem_size);
        /* Сравнение элемента с родительским */
        if (func->less(data + b, data + a, args))
            break;
        do_swap(data + a, data + b, elem_size, swp, args);
        a = b;
    }
}
```

$$T_{\text{Insert}} = O(\log n)$$

Дальнейшее чтение

- [DSABook] Глава 14. «Бинарные кучи»
- [CLRS, Глава 6] Анализ вычислительной сложности алгоритма построения бинарной кучи (*BuildMaxHeap*) за время $O(n)$
- Бинарная куча в ядре Linux: `<linux/min_heap.h>`